

İŞLETİM SİSTEMLERİ

FİNAL SINAVI ÇALIŞMA NOTU

Ders slaytları (OS 1-11) ve hocanın paylaştığı senkronizasyon kodlarının tamamından derlenmiş kapsamlı özet

1. İşletim Sistemlerine Giriş ve Genel Bakış
2. İşlem (Process) Yönetimi ve CPU Sanallaştırma
3. CPU Zamanlama (Scheduling)
4. Süreçler Arası İletişim (IPC)
5. İş Parçacıkları (Threads)
6. İşletim Sistemleri için C Programlama
7. İşlem Senkronizasyonu
8. Klasik Senkronizasyon Problemleri ve Kodları
9. Deadlock (Kilitlenme)
10. Bellek Yönetimi ve Adresleme
11. Sanal Bellek
12. Dosya Sistemleri ve Disk Yönetimi
13. Hızlı Tekrar: Formüller ve Kritik Tablolar

1 | İşletim Sistemlerine Giriş ve Genel Bakış

Bir **işletim sistemi (İS / OS)**, kullanıcı ile bilgisayar donanımı arasında **aracı (arayüz/köprü)** olarak çalışan temel yazılımdır. Bilgisayarda her zaman çalışan temel programdır; diğer tüm uygulama programları işletim sisteminin üzerinde çalışır. Amacı, kullanıcının programları **rahatça ve verimli** bir şekilde çalıştırabileceği bir ortam sağlamaktır. Donanım dünyası (CPU, RAM, depolama, G/Ç birimleri) ile kullanıcı dünyası (program çalıştırma, dosya yönetimi, internet, veri güvenliği) arasındaki boşluğu doldurur.

1.1 Temel ve İkincil Hedefler

Hedef	Açıklama
Kullanıcı Kolaylığı	Kullanıcı dostu bir arayüz sunarak sistemle etkileşimi basitleştirmek.
Kaynak Yönetimi	CPU, bellek, disk gibi kaynakları adil kullanım sağlayacak şekilde yönetmek ve tahsis etmek.
Program Çalıştırma	Kullanıcı programlarının çalışması için gerekli ortamı ve hizmetleri sağlamak.
Güvenlik	Sistemi ve verileri yetkisiz erişime karşı korumak; gizlilik, bütünlük, kullanılabilirlik.
Verimlilik (ikincil)	CPU, bellek ve G/Ç kullanımını/performansını en üst düzeye çıkarmak.
Güvenilirlik (ikincil)	Hataları sorunsuz yönetmek; modüler tasarım, kolay hata ayıklama.

1.2 Çekirdek (Kernel) ve Kabuk (Shell)

- **Çekirdek (Kernel):** İşletim sisteminin temel bileşeni; donanım ile yazılım arasındaki **birincil arayüz**. Sorumlulukları: **Süreç Yönetimi, Bellek Yönetimi, Dosya Sistemi Kontrolü, Aygıt Yönetimi**.
- **Kabuk (Shell):** İşletim sisteminin **en dış katmanı**; kullanıcı etkileşimini yönetir. Kullanıcıdan gelen girdiyi İS için yorumlar, İS'den gelen çıktıyı kullanıcıya sunar. Katman sırası: **Kullanıcı → Shell → Kernel → Donanım**.
- **İki etkileşim yolu:** **CLI** (komut satırı; Bash, PowerShell) ve **GUI** (pencere/ikon/menü; Windows masaüstü, macOS Finder).

1.3 İşletim Sisteminin Kritik Fonksiyonları

- **Uygulama platformu olma:** Diğer programların üzerinde çalışabileceği bir platform sağlar.
- **Giriş-Çıkış yönetimi:** Monitör, klavye, yazıcı gibi kaynakları yönetir.
- **Çoklu görev (Multitasking):** Birden fazla programın kendi bellek alanlarında aynı anda çalışmasına izin verir.
- **Bellek ve dosya yönetimi:** Ana belleği ve ikincil depolamayı yönetir; görevlere bellek atar ve geri alır.
- **Güvenlik sağlama:** Yetkilendirme süreçleriyle sistemi ve uygulamaları güvende tutar.

1.4 Yaygın İşletim Sistemleri ve Felsefeleri

İS	Geliştirici	Tasarım Felsefesi / Kullanım Alanı
Windows	Microsoft	Kişisel bilgisayar, iş ortamı ve oyun için geniş uyumluluk ve esneklik.
macOS	Apple	Yaratıcı endüstriler (tasarım, video, müzik); bütünlük ve kullanıcı dostu deneyim.

İS	Geliştirici	Tasarım Felsefesi / Kullanım Alanı
Linux	Linus Torvalds + açık kaynak	Sunucular, veri merkezleri, geliştirme; özelleştirilebilir ve güçlü.
UNIX	AT&T Bell Labs	Sunucu, iş istasyonu, akademik araştırma; kararlı ve güvenilir temel.

İS seçiminde kilit faktörler: Fiyat (Linux ücretsiz; Windows/macOS ücretli), Uyumluluk (uygulama desteği), Erişilebilirlik/kullanım kolaylığı (macOS kolay, Linux daha karmaşık olabilir), Güvenlik.

Tarihsel tartışma: Andrew **Tanenbaum** → MINIX (eğitim amaçlı, **mikroçekirdek**); Linus **Torvalds** → Linux (MINIX'ten ilham aldı ama **monolitik** yapıyı seçti).

1.5 İşletim Sisteminin Üç Temel Parçası (OSTEP Yaklaşımı)

Ders kitabına (Operating Systems: Three Easy Pieces) göre işletim sistemi üç temel kavram üzerine kuruludur:

a) Sanallaştırma (Virtualization)

- **Tanım:** Fiziksel bir kaynağı alıp onu çok sayıda **sanal kaynağa** dönüştürme işlemi.
- **Amaç:** Her programa, bilgisayarın tüm kaynaklarına tek başına sahipmiş **yanılsamasını (illusion)** vermek.
- **Önemli:** Bu, VirtualBox gibi bir yazılım değildir; işletim sisteminin **kendi çekirdek görevidir**.
- **CPU sanallaştırma kanıtı:** Tek işlemcide 4 program (A, B, C, D) aynı anda çalışıyormuş gibi görünür. **Mekanizma:** milisaniyeler içinde program değiştirme (context switch). **Politika:** sırada hangi programın çalışacağına karar veren zekâ (scheduling).
- **Bellek sanallaştırma kanıtı:** İki program aynı sanal adresi (örn. 0x200000) kullanır ama verileri karışmaz. Her programın kendine ait **Sanal Adres Uzayı** vardır; İS, fiziksel belleği sanal adreslere **haritalar (mapping)**.

b) Eş Zamanlılık (Concurrency)

- Sadece İS'nin değil, tüm modern yazılımların (veritabanı, web sunucusu) ortak sorunudur.
- **Deney:** İki thread aynı sayacı 100.000 kez artırır. Beklenen: 200.000; gerçek sonuç: 143.052 gibi rastgele bir değer → **Yarış Durumu (Race Condition)**.
- **Sebebi:** counter = counter + 1 tek bir işlem değildir; assembly'de 3 adımdır: **LOAD** (bellekten al) → **INCREMENT** (artır) → **STORE** (belleğe yaz). Bu adımlar **atomik (bölünemez)** olmadığı için araya context switch girerse veri tutarsızlığı oluşur. Çözüm: kilitler ve semaforlar (Bölüm 7).

c) Kalıcılık (Persistence)

- **DRAM (ana bellek)** uçucudur (volatile): elektrik kesilince veri yok olur. **Disk/SSD** kalıcıdır (persistent).
- Yazılım katmanı: **Dosya Sistemleri**. İS, güvenilmez donanımı güvenilir bir veri deposuna dönüştürür.
- **Soyutlama sanatı:** Kullanıcı API'si basittir (open, write, close); altta izin kontrolü, sektör hesabı, denetleyici komutları gibi karmaşık, donanım bağımlı detaylar vardır. **Cihaz sürücüler (device drivers)** bu karmaşıklığı maskeler.

Bir İS'nin tasarım hedefleri: **Soyutlama** (karmaşık donanımı kolay arayüze çevirme), **Koruma/İzolasyon** (bir programın hatası tüm sistemi çökertmesin), **Performans** (ek yükü/overhead'i minimize et), **Güvenilirlik & Verimlilik** (7/24 kesintisiz çalışma, enerji tasarrufu).

1.6 İşletim Sistemi Türleri

İşlevsellik bazında

Tür	Özellikler	Örnek
Single Tasking	Aynı anda tek işlem; bellekte tek program; biri bitmeden diğeri başlayamaz.	Eski MS-DOS
Multiprogramming → Multitasking	Bellekte birden fazla işlem; CPU zaman paylaşımı yapar; amaç CPU'nun boş kalmaması.	Windows, Linux, macOS
Multithreading	Tek process içinde birden fazla thread; thread'ler aynı bellek alanını paylaşır.	Tarayıcı: bir thread UI, diğeri indirme
Multiprocessing	Birden fazla CPU/çekirdek; gerçek paralellik.	Çok çekirdekli sistemler, sunucular
Multiuser	Aynı anda birden fazla kullanıcı; ayrı oturum ve yetkiler.	Linux/Unix sunucular

- **Thread (iş parçacığı):** CPU tarafından çalıştırılabilen **en küçük yürütme birimidir**. Aynı process içindeki thread'ler kodu, heap'i ve dosyaları paylaşır; context switch maliyeti process'e göre **daha düşüktür**.

Amaç bazında

Tür	Örnek
Batch (Yığın) İS	Kütüphane kitap kayıtları, market stok raporları
Distributed (Dağıtık) İS	LOCUS, MICROS, Amoeba
Network (Ağ) İS	Microsoft Windows Server 2025, macOS Server
Real-Time (Gerçek Zamanlı) İS	Medikal görüntüleme sistemleri, robotlar
Mobil İS	Android, iOS, HarmonyOS

2 | İşlem (Process) Yönetimi ve CPU Sanallaştırma

2.1 İşletim Sisteminin İki Temel Rolü

- **1. Genişletilmiş Sanal Makine:** Karmaşık ve kullanımı zor donanımı, kullanışlı ve soyutlanmış kütüphanelere/API'lere dönüştürür.
- **2. Kaynak Yöneticisi:** İşlemci, bellek, disk gibi sınırlı kaynakları çok sayıda program arasında verimli ve adil paylaşır.

İlk işletim sistemleri (mainframe dönemi) sadece kütüphane görevi görüyordu; modern sistemler **koruma (protection)** ve **çoklu programlama** ile tam bir kaynak yöneticisidir. **Eş zamanlılık yanılması:** İS bir işlemi kısa süre çalıştırır, durdurur, diğerini çalıştırır; saniyede binlerce kez gerçekleşen bu değişim süreklilik illüzyonu yaratır. **Bedel:** performans kaybı (overhead) — ama çoklu görev için zorunludur.

2.2 Program ve İşlem (Process) Farkı

- **Program:** Diskte duran, **pasif ve ölü** bir varlık (bayt yığını).
- **İşlem (Process):** Çalışan program. Bellek alanı (adres uzayı), yazmaçlar (registers) ve yığın (stack) ile canlanmış **aktif** varlık.

İşlem Adres Uzayı (Process Address Space)

- **Yüksek adresten düşük adrese: Stack (Yığın)** → aşağı doğru büyür (yerel değişkenler, fonksiyon parametreleri, dönüş adresleri).
- **Boş alan** ortadadır; **Heap (Öbek)** → yukarı doğru büyür (dinamik bellek: malloc/free).
- **Data (Veri)** → global/statik değişkenler; **Text/Code (Kod)** → program kodu (en düşük adresler).

2.3 İşlem Oluşturma Adımları

- 1) Program kodu diskten işlemin adres alanına **yüklenir (LOAD)**. Modern İS bunu **tembel (lazy)** yapar: kod/veri parçaları yalnızca yürütme sırasında gerekirse yüklenir.
- 2) Çalışma zamanı **yığını (stack)** tahsis edilir ve `main()`'in `argc/argv` argümanlarıyla başlatılır.
- 3) **Heap** oluşturulur: C'de `malloc()` ile alan istenir, `free()` ile boşaltılır.
- 4) **G/Ç kurulumu:** varsayılan olarak her işlemin **3 açık dosya tanıtıcısı (file descriptor)** vardır: standart girdi (0), çıktı (1), hata (2).
- 5) Giriş noktasında — `main()` — program başlatılır; İS, CPU kontrolünü yeni işleme aktarır.

2.4 İşlem Durumları ve Yaşam Döngüsü

Durum	Ne zaman olur?	Açıklama
New (Yeni)	Program RAM'e yükleniyor	PCB oluşturulur; stack/heap ayrılır.
Ready (Hazır)	Yükleme tamam, yürütmeye hazır	Ready queue'da CPU sırası bekler.
Running (Çalışıyor)	Scheduler işlemi seçti	PCB'deki register değerleri CPU'ya aktarıldı; işlem CPU üzerindedir.
Blocked / Waiting	I/O veya kaynak bekliyor	CPU bırakıldı; register'ların anlık görüntüsü (snapshot) PCB'ye kaydedildi.
Terminated (Sonlandı)	İş bitti	Bellek temizlendi; PCB girdisi kaldırıldı.

Geçişler: New → Ready (kabul); Ready → Running (**scheduler seçimi**); Running → Ready (**zamanlayıcı kesmesi**); Running → Blocked (**I/O isteği / bekleme**); Blocked → Ready (**I/O tamamlandı**); Running → Exit/Terminated.

2.5 İşlem Kontrol Bloğu (PCB)

PCB, işlemin '**beyni**' ve adeta '**save game**' dosyasıdır: yaşam döngüsündeki işlemlerin hayati bilgileri burada saklanır ve **korunmalı çekirdek alanında (protected kernel space)** tutulur. Tipik alanlar: **pid** (kimlik), **state** (durum), **kstack** (kernel stack pointer), **context** (yazmaçlar), **açık dosyalar**, **bellek boyutu (sz)**, **parent** (ebeveyn işlem).

xv6 – proc yapısı

```
// xv6 işletim sistemi - struct proc (her işlem için tutulan bilgi)
struct proc {
    char *mem;           // İşlem belleğinin başlangıcı
    uint sz;             // İşlem belleğinin boyutu
    char *kstack;        // Bu işlem için çekirdek yığınının tabanı
    enum proc_state state; // İşlem durumu
    int pid;             // İşlem kimliği (Process ID)
    struct proc *parent; // Ebeveyn işlem
    void *chan;          // Sıfır değilse: chan üzerinde uyuyor
    int killed;          // Sıfır değilse: öldürülmüş
    struct file *ofile[NOFILE]; // Açık dosyalar
    struct inode *cwd;    // Geçerli dizin
    struct context context; // İşlemi çalıştırmak için buraya geçilir
    struct trapframe *tf; // Geçerli kesme için trap çerçevesi
};
// Kaydedilip geri yüklenen yazmaçlar:
struct context { int eip, esp, ebx, ecx, edx, esi, edi, ebp; };
// xv6'da işlem durumları:
enum proc_state { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
```

2.6 Sınırlı Doğrudan Yürütme (Limited Direct Execution)

İkilem: İS, programı doğrudan CPU'da çalıştırırsa **en yüksek performansı** alır ama **kontrolü kaybeder** (program sonsuz döngüye girebilir veya makineyi ele geçirebilir). Çalışan programlara hiçbir sınırlama konmazsa İS hiçbir şeyi kontrol edemez ve **sadece bir kütüphane** olur.

Çözüm: Sınırlı Doğrudan Yürütme — program doğrudan CPU'da çalışır, ancak kritik anlarda kontrolü İS'ye iade eden **donanımsal kilitler** kullanılır: kullanıcı/çekirdek modları + trap mekanizması + zamanlayıcı kesmesi.

Kullanıcı Modu ve Çekirdek Modu

	User Mode (Kullanıcı Modu)	Kernel Mode (Çekirdek Modu)
Tanım	Uygulamaların çalıştığı kısıtlı yetki seviyesi	İS çekirdeğinin çalıştığı tam yetkili mod
Donanım erişimi	Doğrudan erişemez	Doğrudan erişebilir
Bellek	Yalnızca kendine ayrılan kısım	Belleğin tamamı
Hata olursa	Genellikle sadece ilgili uygulama çöker	Tüm sistem çökebilir (mavi ekran)
Örnek	Word, Chrome, Python programınız	Linux çekirdeği, ekran kartı / disk sürücüler

Sistem Çağrısı (System Call) ve Trap

- Kullanıcı programının çekirdekten **hizmet istemek** (dosya okuma, işlem oluşturma/yok etme, işlemlerle iletişim, bellek tahsisi vb.) için kullandığı **köprüdür**.
- trap** buyruğu: çekirdeğe atla + ayrıcalık seviyesini **çekirdek moduna yükselt**.

- **return-from-trap** buyruğu: çağırın programa geri dön + ayrıcalığı **kullanıcı moduna düşür**.
- **İS, boot sırasında trap tablosunu** kurar: syscall işleyicisinin ve zaman kesmesi işleyicisinin adreslerini donanıma bildirir.

Kontrolü Geri Kazanmak: İşlemler Arası Geçiş

- **İşbirlikçi (cooperative) yaklaşım:** İşlemler yield gibi sistem çağrılarıyla CPU'yu düzenli bırakır; hata yapınca da (sıfıra bölme, izinsiz bellek erişimi) kontrol İS'ye geçer. **Risk:** sonsuz döngüye giren bir program sistemi kilitler → tek çare makineyi yeniden başlatmak.
- **İşbirlikçi olmayan yaklaşım — Zamanlayıcı Kesmesi (Timer Interrupt):** Donanım her birkaç milisaniyede bir kesme üretir; çalışan program **zorla durdurulur**, durum bilgileri kaydedilir ve kontrol İS'nin **kesme işleyicisine (interrupt handler)** geçer. Böylece İS her zaman 'patron koltuğundadır'.
- **Eşzamanlılık endişesi:** Kesme işlenirken başka kesme gelirse? İS, kesme işlenirken kesmeleri devre dışı bırakabilir veya kendi veri yapılarına erişirken sofistike kitleme yöntemleri kullanır.

2.7 Bağlam Değiştirme (Context Switch)

Scheduler başka bir işleme geçmeye karar verirse **context switch** gerçekleştirilir. Bu sırada CPU faydalı kod çalıştırmaz (**overhead**), ancak çoklu görev için zorunludur. Düşük seviye (assembly) adımlar:

- 1) Mevcut işlemin yazmaçlarını (genel amaçlı yazmaçlar, PC, kernel SP) **kernel stack'e kaydet**.
- 2) Mevcut kernel SP'yi **PCB'ye yaz**.
- 3) Yeni işlemin kernel SP'sini **yükle** (yığın burada değişir).
- 4) Yeni işlemin yazmaçlarını kernel stack'ten **geri yükle**.
- 5) **iret** (interrupt return) ile user PC ve user SP yüklenerek **kullanıcı moduna dönülür**.

Timer interrupt'lı tam protokol: Zaman kesmesi gelir → donanım, A'nın yazmaçlarını çekirdek yığına kaydeder, çekirdek moda geçer, trap işleyiciye atlar → İS trap'ı işler, switch() rutinini çağırır: regs(A)'yi proc-struct(A)'ya kaydet, regs(B)'yi proc-struct(B)'den yükle, çekirdek yığın(B)'ye geç → return-from-trap(B) → donanım regs(B)'yi yükler, kullanıcı moduna geçer, B'nin PC'sine atlar.

2.8 İşlem API'si: fork(), wait(), exec()

fork() — işlem kopyalama

- Çağırın işlemin **birebir kopyasını** oluşturur. Dönüş değeri (rc): **çocukta 0, ebeveynde çocuğun PID'i (>0)**, hata durumunda <0.
- fork() sonrası ebeveyn mi çocuk mu önce çalışır **belirsizdir (non-deterministic)**; çıktı sırası her çalıştırmada değişebilir.

wait() — belirsizliği yönetmek

- Ebeveyn, **çocuk tamamlanana kadar kendini bloklar**; çocuk çalışır ve sonlanır; ebeveyn uyanır ve devam eder → akış **deterministik** olur. wait(), çocuğun **PID'ini döndürür**.

exec() — kimlik değişimi

- Mevcut işlemin belleğini (kod, veri, yığın) **siler** ve yerine diskteki **yeni bir programı yükler**.
- **Dikkat:** Yeni bir işlem **yaratılmaz**, PID **değişmez**; aynı işlem kabuğu içinde artık bambaşka bir program çalışır. exec başarılıysa sonrasındaki satırlar **asla çalışmaz**.

fork + exec + wait kalıbı (p3.c)

```

int rc = fork();
if (rc < 0) {                                // fork başarısız
    fprintf(stderr, "fork failed\n"); exit(1);
} else if (rc == 0) {                        // ÇOCUK işlem
    char *myargs[3];
    myargs[0] = strdup("wc");               // çalıştırılacak program
    myargs[1] = strdup("p3.c");             // argüman
    myargs[2] = NULL;                       // dizinin sonu
    execvp(myargs[0], myargs);              // işlemi 'wc' ile değiştir
    printf("bu satır yazdırılmamalı"); // exec başarılıysa çalışmaz
} else {                                    // EBEVEYN işlem
    int wc = wait(NULL);                    // çocuğu bekle (deterministik akış)
}

```

Neden fork ve exec ayrı tasarlandı? (Unix felsefesi) Bu ayrım, kabuğun (shell) **fork ile exec arasındaki küçük zaman diliminde ortamı değiştirmesine** olanak tanır. Örn. wc file.txt > output.txt: shell fork yapar; çocukta stdout kapatılıp output.txt açılır; sonra exec('wc') çağrılır → çıktı dosyaya yazılır. **G/Ç yönlendirme (redirection) ve boru hatları (pipes)** bu tasarım sayesinde mümkündür.

✓ **BÖLÜM ÖZETİ:** Sanallaştırma = zaman paylaşımıyla tek CPU çok CPU gibi davranır. Soyutlama = Process (PCB + bellek + durum). Güvenlik = user/kernel modları + trap. Kontrol = timer interrupt. API = fork, exec, wait. İS, her programa 'tüm makine senin' yalanını söyleyen ama mutlak kontrolü elinde tutan bir illüzyonisttir.

3 | CPU Zamanlama (Scheduling)

3.1 Scheduler ve Dispatcher

	Scheduler (Planlayıcı)	Dispatcher (Görevlendirici)
Görevi	Hangi sürecin CPU'yu kullanacağına KARAR verir; hazır kuyruğundaki süreçleri değerlendirir; planlama politikasını uygular.	Scheduler'ın seçtiği süreci CPU üzerinde GERÇEKTEN çalıştırır; context switch yapar; kullanıcı moduna geçişi sağlar.
Özet	Karar verme mekanizması	Kararı uygulayan mekanizma

Scheduler Türleri

- **Long-Term (Uzun Dönem):** Diskteki işlerden hangilerinin belleğe (ready queue'ya) alınacağını seçer; çoklu programlama derecesini belirler.
- **Short-Term (Kısa Dönem / CPU Scheduler):** Ready queue'dan hangi sürecin CPU'ya gideceğini seçer; en sık çalışan planlayıcıdır.
- **Medium-Term (Orta Dönem):** I/O bekleyen / askıya alınan süreçlerle ilgilenir (bellekten çıkarma-geri alma, swap).

3.2 Varsayımlar ve Zamanlama Ölçütleri

Başlangıç iş yükü varsayımları (sonra teker teker gevşetilir): 1) Her iş **aynı süre** çalışır. 2) Tüm işler **aynı anda** gelir. 3) İşler yalnızca **CPU** kullanır (I/O yapmaz). 4) Her işin **çalışma süresi önceden bilinir**.

- **Geri dönüş süresi (Turnaround Time)** = İşin tamamlandığı an – İşin sisteme geldiği an. (Performans ölçütü)
- **Tepki süresi (Response Time)** = İşin ilk kez zamanlandığı an – Geldiği an. (Etkileşim ölçütü)
- **Bekleme süresi (Waiting Time)** = Turnaround – Burst (hazır kuyruğunda geçen toplam süre).
- **Adalet (Fairness):** Performans ile adalet, zamanlamada genellikle **birbiriyle çelişir**.

3.3 FIFO / FCFS ve Konvoy Etkisi

- **FIFO (First In First Out) / FCFS (First Come First Served):** İlk gelen ilk hizmet alır. Basit ve kolay gerçekleştirilir.
- Örnek: A, B, C aynı anda gelir, her biri 10 sn → ortalama turnaround $(10+20+30)/3 = 20$ sn. Sorun yok.
- **Konvoy etkisi:** A=100 sn, B=C=10 sn ve A burun farkıyla önce gelirse: turnaround $(100+110+120)/3 = 110$ sn. Kısa işler, uzun işin arkasında **konvoy gibi** bekler.

3.4 SJF ve STCF

- **SJF (Shortest Job First / Önce En Kısa İş):** En kısa işi önce çalıştır. **Boşaltmasız (non-preemptive)**. Aynı anda gelen A=100, B=C=10 için: B, C, A → ortalama turnaround $(10+20+120)/3 = 50$ sn.
- **SJF'nin zaafı:** B ve C, t=10'da gelirse A'nın bitmesini beklemek zorundadırlar (non-preemptive).
- **STCF (Shortest Time-to-Completion First) / Boşaltmalı SJF (SRTF):** SJF'ye **boşaltma (preemption)** eklenir. Yeni iş geldiğinde, yeni işin süresi ile mevcut işlerin **kalan** süreleri karşılaştırılır; en kısa olan çalıştırılır. Örnekte A kesilir, B ve C biter, A devam eder.

3.5 Öncelik Zamanlaması (Priority Scheduling) — Örnek Hesaplı

Veri: P1 (geliş 0, burst 4, öncelik 2), P2 (geliş 1, burst 2, öncelik 1 — en yüksek), P3 (geliş 2, burst 6, öncelik 3).

Non-Preemptive (Boşaltmasız)

Sıra: P1 (0→4) bitene kadar çalışır; sonra en öncelikli olan P2 (4→6); sonra P3 (6→12).

İşlem	Completion	Turnaround (CT–Geliş)	Waiting (TAT–Burst)
P1	4	4	0
P2	6	5	3
P3	12	10	4

Ortalama Bekleme = $(0+3+4)/3 = 7/3$; Ortalama Turnaround = $(4+5+10)/3 = 19/3$.

Preemptive (Boşaltmalı)

Sıra: P1 (0→1); t=1'de P2 gelir (öncelik 1 < 2) → P1 kesilir, P2 (1→3); P1 devam (3→6); P3 (6→12).

İşlem	Completion	Turnaround	Waiting
P1	6	6	2
P2	3	2	0
P3	12	10	4

Ortalama Bekleme = $(2+0+4)/3 = 2$; Ortalama Turnaround = $(6+2+10)/3 = 6$.

► **SINAV NOTU:** Bu iki tablo, sınavda 'Gantt şeması çizip ortalama bekleme/turnaround hesaplayın' tipi sorunun birebir örneğidir. Formülleri ezberle: **TAT = Completion – Arrival**, **WT = TAT – Burst**.

3.6 Round Robin (RR)

- **Dilimlemeli zamanlama:** Bir işi bir **zaman dilimi (time slice / scheduling quantum)** kadar çalıştır, sonra kuyruktaki bir sonrakine geç; tüm işler bitene kadar döngü sürer.
- Zaman diliminin uzunluğu **saat kesmesi süresinin tam katı** olmalıdır.
- STCF/SJF tepki süresinde kötüdür; **RR tepki süresinde çok iyidir** (örn. A, B, C 5'er sn, kuantum 1 sn → herkes ilk 3 sn içinde başlar).
- **RR adildir; ancak turnaround gibi ölçütlerde performansı düşüktür** (işleri uzatır).
- **Kuantum ödünleşmesi:** Kısa kuantum → iyi tepki süresi ama context switch maliyeti artar; uzun kuantum → switch maliyeti amortize olur ama tepki süresi kötüleşir. Karar bir **trade-off** sorunudur.

3.7 I/O'yu Dikkate Almak (Örtüşme / Overlap)

- Bir iş I/O isteği yapınca **bloklanır**; zamanlayıcı **başka bir işi** çalıştırır. I/O bitince **kesme** gelir; İS bloklanan işi **ready** durumuna geçirir.
- Örnek: A toplam 50 ms CPU ister ama her 10 ms'de bir 10 ms I/O yapar; B 50 ms saf CPU işidir. A'nın I/O'ları sırasında B çalıştırılırsa kaynaklar **örtüşür** ve sistem verimli kullanılır → **işlemci kullanımını maksimize et**.

3.8 Çok Seviyeli Geribeslemeli Kuyruk (MLFQ)

Problem: İşlerin çalışma süresi **bilinmiyor**. **Fikir:** Geçmişe bakarak geleceği tahmin et. **Amaç:** Hem turnaround'ı (kısa işleri öne al) hem tepki süresini optimize et — süreyi bilmeden.

Birden çok öncelik kuyruğu vardır; iş davranışına göre kuyruklar arasında taşınır:

- **Kural 1:** Öncelik(A) > Öncelik(B) ise A çalışır, B çalışmaz.
- **Kural 2:** Öncelik(A) = Öncelik(B) ise A ve B Round-Robin ile çalışır.
- **Kural 3:** Sisteme yeni gelen iş **en yüksek öncelikli** kuyruğa konur.
- **Kural 4 (iyileştirilmiş):** Bir iş herhangi bir seviyede kendisine verilen **tüm zaman dilimini kullanırsa** (CPU'yu kaç kere bıraktığından **bağımsız**), önceliği bir düşürülür (alt kuyruğa iner).
- **Kural 5 (Öncelik Artırma / Priority Boost):** Belirli bir **S** süresi sonunda sistemdeki **tüm işler en üst kuyruğa** taşınır.

Temel MLFQ'nun problemleri ve çözümleri

- **Açlık (Starvation):** Çok sayıda interaktif iş varsa, uzun işler hiç CPU alamaz → **çözüm Kural 5** (boost).
- **Zamanlayıcıyı aldatma (gaming):** Kuantumun %99'unu kullanıp I/O yaparak hep üst kuyrukta kalma hilesi → **çözüm: daha iyi hesap tutma** (Kural 4'ün yeni hali: toplam kullanım sayılır).
- **Davranış değişimi:** CPU-yoğun bir iş sonradan interaktif olabilir → boost bunu da çözer.
- **Kuantum ayarı:** Yüksek öncelikli kuyruklar **kısa** kuantum (örn. ≤ 10 ms), düşük öncelikliler **uzun** kuantum (örn. 100 ms). Örnek: en üst 10 ms, orta 20 ms, en alt 40 ms.
- **Gerçek örnek — Solaris:** 60 kuyruk; kuantum 20 ms'den birkaç yüz ms'ye yavaşça artar; her 1 saniyede bir öncelik artırma.
- MLFQ, davranışı sayesinde **SJF gibi davranmayı öğrenir:** I/O yapıp CPU'yu erken bırakan (interaktif) işler üstte kalır.

3.9 Orantılı Pay (Proportional / Fair-Share) Zamanlama

Her işin CPU zamanını **belirli bir oranda** kullanması garanti edilir. Turnaround/tepki süresi açısından optimum değildir; amaç **adalettir**. Temel kavram: **bilet (ticket)** — bir işlemin kaynaktan alacağı payı temsil eder. Örn. A=75 bilet → CPU'nun %75'i; B=25 bilet → %25'i.

Piyango (Lottery) Zamanlama

- Zamanlayıcı her dilimde **rastgele bir kazanan bilet** çeker; bileti hangi işlem tutuyorsa CPU'yu o kullanır. Örn. 100 bilet: A=0-74, B=75-99.
- İşler ne kadar **uzun** yarışırsa orana o kadar yaklaşılar; işler kısaysa **adaletsizlik ciddi** olabilir (olasılıksal yöntem).

Lottery – implementasyon

```
int counter = 0;
int winner = getrandom(0, totaltickets); // 0..toplam bilet arası rastgele
node_t *current = head; // iş listesinin başı
while (current) {
    counter = counter + current->tickets;
    if (counter > winner) break; // kazanan bulundu
    current = current->next;
}
// 'current' kazanandır: onu zamanla
```

Adımlı (Stride) Zamanlama — deterministik adil pay

- **Adım (stride)** = Büyük bir sayı / işlemin bilet sayısı. Örn. 10.000 için: A(100 bilet)→100, B(50)→200, C(250)→40.
- Her işlemin bir **pass (sayaç)** değeri vardır. Daima **en düşük pass** değerli işlem çalıştırılır; çalışınca pass değeri kendi stride'ı kadar artırılır.

Stride – döngü

```
current = remove_min(queue);      // en düşük pass'lı işlemi seç
schedule(current);                // kuantum boyunca çalıştır
current->pass += current->stride;  // pass'ı stride kadar artır
insert(queue, current);           // kuyruğa geri koy
```

- **Problem:** Sonradan gelen yeni bir işlemin pass'ı 0 olur ve CPU'yu uzun süre tekeline alabilir (lottery'de bu sorun yok).

Linux CFS (Completely Fair Scheduler)

- Adil payı farklı sağılar ve zamanlama kararına **çok az zaman** harcamayı hedefler (veri merkezlerinde scheduler, CPU süresinin ~%5'ini yiyebilir!).
- Her işlem çalışırken **vruntime** (sanal çalışma süresi) değeri artar; karar anında **en düşük vruntime**'lı işlem seçilir.
- **sched_latency** değeri ile bir işlemin geçiş öncesi ne kadar çalışacağı belirlenir (zaman dilimi **dinamiktir**).
- **nice mekanizması:** -20 ile +19 arası (varsayılan 0). CFS, nice'ı bir **ağırlığa (weight)** eşler ve vruntime'ı ağırlıklı hesaplar → bazı işlemlere daha fazla CPU payı verilebilir.
- Hazır işler **kırmızı-siyah ağaçta** vruntime'a göre sıralı tutulur: ekleme/silme **O(log n)** (sıralı listede O(n) olurdu; binlerce işlemde fark büyüktür).
- **Uyuyan işlem sorunu:** 10 sn uyuyan B uyanınca vruntime'ı çok geride kalır ve CPU'yu tekeline alıp A'yı aç bırakabilir. **Çözüm:** Uyanan işin vruntime'ı **ağaçtaki minimum değere eşitlenir** (ağaçta yalnızca çalışmaya hazır işler vardır).

4 | Süreçler Arası İletişim (IPC)

4.1 Süreçler Neden İşbirliği Yapar?

- **Bilgi paylaşımı:** dosyalar, istekler vb.
- **Hesaplama hızını artırma:** çok işlemcili ortamlarda bir görevin farklı kısımları paralel çalıştırılabilir.
- **Modülerlik:** sistemi temiz ve düzenli uygulamak için modüllere / süreçlere / parçacıklara bölmek.

İşbirliği yapan süreçler **aynı makinede** (bu dersin konusu) veya **farklı makinelerde** (ağ üzerinden — dağıtık sistemler, ders kapsamı dışı) çalışabilir. İki temel konu: **1)** Veri paylaşımı nasıl gerçekleşir? **2)** Tutarlılık için süreçler nasıl **senkronize** edilir?

4.2 Veri Paylaşımı Yöntemleri

- **Paylaşımlı Bellek (Shared Memory):** İS, farklı süreçlerin **aynı bellek alanını kendi adres uzaylarına eşlemesine** izin verir. Paylaşılan tüm veriler ortak alanda tutulur; bu belleği paylaşan tüm süreçler erişebilir.
- **Mesajlaşma (Message Passing):** pipe, mailbox, socket vb. araçlarla veri alışverişi.
- **Thread'ler için not:** Aynı süreçteki thread'ler **yığın (stack) ve yazmaçlar dışında** hemen her şeyi paylaşır; bu yüzden thread'ler arası iletişim çok daha kolaydır.

4.3 Mesajlaşma (Message Passing)

- İki temel işlem: **send(message)** ve **receive(message)**.
- P ve Q iletişicekse: 1) aralarında bir **iletişim bağlantısı (link)** kurulmalıdır; 2) bu bağlantı genellikle **çekirdek tamponlarında (kernel buffer)** mantıksal bir bağlantıdır; 3) mesajlar send/receive ile aktarılır.
- **Kernel buffer:** İS'nin çekirdek alanında tuttuğu geçici veri depolama alanıdır; amacı süreçler veya cihazlar arasında veri aktarımını **güvenli ve düzenli** yapmaktır.
- **Gerçekleştirme soruları:** Bağlantılar nasıl kurulur? Bir bağlantı ikiden fazla süreçle ilişkili olabilir mi? İki süreç arasında kaç bağlantı olabilir? Kapasite nedir? Mesaj boyutu sabit mi değişken mi? Tek yönlü mü çift yönlü mü? (Örnek: **Pipe → tek yönlü; Unix Message Queue → çift yönlü.**)

Doğrudan İletişim

- Süreçler birbirini **açıkça belirtir**: send(P, msg) → P'ye gönder; receive(Q, msg) → Q'dan al.
- Bağlantı **otomatik** kurulur; her bağlantı **yalnızca bir süreç çiftiyle** ilişkilidir; her çift arasında **tek bağlantı** vardır; bağlantı genellikle **çift yönlüdür**.

Dolaylı İletişim (Mailbox)

- Mesajlar **mailbox (posta kutusu)** aracılığıyla gönderilir/alınır: send(A, msg), receive(A, msg) — A bir mailbox kimliğidir.
- Her mailbox **benzersiz bir kimliğe** sahiptir; süreçler ancak **aynı mailbox'ı paylaşıyorsa** iletişebilir.
- Bir bağlantı **birden fazla süreçle** ilişkili olabilir; süreçler birden fazla bağlantı paylaşabilir; bağlantı tek veya çift yönlü olabilir. Mailbox işlemleri: oluşturma, mesaj gönderme/alma, silme.

Kernel Buffer Kapasitesi (3 uygulama biçimi)

Kapasite	Davranış
Sıfır kapasite	Kuyrukta mesaj tutulamaz. Gönderen alıcıyı (ve tersi) bekler → randevu (rendezvous).
Sınırlı kapasite	Kuyruk doluyorsa gönderen bekler; kuyruk boşsa alıcı bekler.
Sınırsız kapasite	Gönderen hiçbir zaman beklemez; kuyruk boşsa alıcı bekler.

► **SINAV NOTU:** Bu tampon mantığı, Bölüm 8'deki **Üretici-Tüketici (Producer-Consumer)** problemine doğrudan bağlanır: sınırlı tamponda 'doluyorsa üretici bekler, boşsa tüketici bekler' davranışı semaforlarla gerçekleştirilir.

5 | İş Parçacıkları (Threads)

5.1 İşlemin İki Özelliği ve Thread Kavramı

- **Kaynak Sahipliği:** İşlem, işlem görüntüsünü tutan **sanal bir adres alanı** içerir; İS, işlemler arasında istenmeyen etkileşimi önlemek için **koruma** işlevi görür.
- **Planlama / Yürütme:** İşlem bir yürütme yolunu izler; yürütme durumuna ve önceliğe sahiptir; İS tarafından zamanlanan/dağıtılan varlıktır.

Modern İS'lerde bu iki özellik **ayrılır**: **Dispatching (zamanlama) birimi = thread** (iş parçacığı, *lightweight process*); **kaynak sahipliği birimi = process** (görev/task). **Multithreading:** İS'nin, tek bir süreç içinde **birden fazla eşzamanlı yürütme yolunu** destekleme yeteneğidir.

- **Tek iş parçacıklı yaklaşım:** İşlem başına tek yürütme ipliği; thread kavramı tanınmaz. Örnek: **MS-DOS**.
- **Çok iş parçacıklı:** Örnek: **Java çalışma zamanı ortamı** (tek süreç, çok thread); modern İS'ler (çok süreç, süreç başına çok thread).

5.2 Thread'e Özel vs. Süreçte Ortak Olanlar

Thread'e ÖZEL (izole)	Süreçte AİT (paylaşılan)
Thread kimliği (TID)	Kod bölümü (text)
Program sayacı (PC)	Veri bölümü ve heap
Yazmaç seti (registers)	Açık dosyalar ve sinyaller
Yığın (stack)	Adres uzayı ve diğer kaynaklar (G/Ç vb.)

Thread Control Block (TCB) içeriği: TID, thread durumu (çalışıyor/hazır/beklemede), PC (son komutun adresi), CPU yazmaçları, stack pointer + base pointer, kesilme/devam noktası (context bilgisi), thread'e özel kaynaklar (TLS — thread-local storage, kilitler vb.).

Bir işlemin **tüm thread'leri aynı adres alanını paylaştığı** için, bir thread'in bir kaynak üzerindeki değişikliği diğer thread'leri de etkiler → **senkronizasyon gerekir** (Bölüm 7).

5.3 Thread'lerin Avantajları — Dört Temel Direk

Direk	Açıklama
1. Duyarlılık (Responsiveness)	Uygulamanın bir kısmı bloke olsa veya uzun iş yapsa bile program çalışmaya devam eder; kesintisiz etkileşim.
2. Kaynak Paylaşımı (Resource Sharing)	Thread'ler ait oldukları sürecin belleğini ve kaynaklarını varsayılan olarak paylaşır; aynı adres alanında kolektif çalışma.
3. Ekonomi (Economy)	Yeni süreç yaratmak bellek/kaynak açısından pahalıdır; thread oluşturma ve thread'ler arası context switch çok daha ucuzdur.

Direk	Açıklama
4. Çok İşlemcili Mimari (Multiprocessor)	Tek süreç tek iş parçacığıyla yalnızca bir CPU kullanır; çoklu thread, eşzamanlılığı artırıp donanım potansiyelini zirveye taşır.

- Diğer kullanım örnekleri (tek kullanıcı sistem): ön plan/arka plan çalışması (bir thread menü + girdi, diğeri komut yürütür), **asen kron işlem** (periyodik yedekleme), **yürütme hızı, modüler program yapısı**.
- Gerçek dünya: **chrome.exe** — bir thread sayfayı çizer (görüntüleme), başka bir thread aynı anda dosya indirir → kullanıcı indirme bitmeden gezinmeye devam eder.

5.4 Thread Durumları ve Süreç Düzeyi İşlemler

- Temel thread durumları: **Running, Ready, Blocked**. Thread işlemleri: **Spawn** (oluşturma), **Block, Unblock, Finish**.
- Zamanlama ve görevlendirme **thread bazında** yapılır; ancak bazı işlemler süreç düzeyindedir: bir işlemi **askıya almak (suspend)** tüm thread'lerini askıya alır; bir işlemi **sonlandırmak** içindeki tüm thread'leri sonlandırır.

5.5 Kullanıcı ve Çekirdek Düzeyi Thread'ler

ULT — Kullanıcı Düzeyi (Many-to-One, M:1)

- Tüm thread yönetimi **uygulama (thread kütüphanesi)** tarafından yapılır; **çekirdek thread'lerden habersizdir** ve işlemi tek birim olarak zamanlar (işleme tek yürütme durumu atar).
- **Dezavantajlar:** 1) Bir ULT **engelleyici (blocking) sistem çağrısı** yaparsa yalnızca o thread değil, **süreçteki tüm thread'ler bloke olur**. 2) Çekirdek bir işlemi aynı anda yalnızca **bir işlemciye** atar → çoklu işleminden (multiprocessing) yararlanılamaz.

KLT — Çekirdek Düzeyi (One-to-One, 1:1)

- Thread yönetimini **çekirdek** yapar; uygulamada yönetim kodu yoktur, yalnızca API vardır. Örnek: **Windows**.
- **Avantajlar:** Aynı işlemin birden fazla thread'i **aynı anda farklı işlemciler**e zamanlanabilir; bir thread bloke olursa çekirdek **aynı işlemin başka thread'ini** çalıştırabilir; çekirdek rutinlerinin kendisi çok iş parçacıklı olabilir.
- **Dezavantaj:** Aynı işlem içinde thread'ler arası geçiş bile **çekirdeğe mod değişikliği** gerektirir. Ölçümler (VAX, Tablo 4.1): Null Fork ve Signal-Wait sürelerinde **ULT ≪ KLT ≪ Process** — aralarında büyüklük mertebesi fark vardır.

Birleşik Yaklaşım (Many-to-Many, M:N)

- Thread oluşturma, zamanlama ve senkronizasyonun büyük kısmı **kullanıcı alanında**; çok sayıda ULT, daha az veya eşit sayıda KLT'ye **çoklanır (multiplexing)**. Örnek: **Solaris**.

Model	Bloklanma Riski	Çoklu Çekirdek	İS Yüğü
Çoktan Teke (M:1)	Yüksek (tüm süreç durur)	Yok (tek işlemciye sıkışır)	Çok düşük
Bire Bir (1:1)	Yok (diğerleri devam eder)	Tam paralellik	Yüksek (sayı kısıtlaması gerektirir)
Çoktan Çoğa (M:N)	Yok (çekirdek dinamik atar)	Tam paralellik	Dengeli ve dinamik

5.6 Hyperthreading (SMT) ve Thread İptali

- **Hyperthreading / SMT:** Fiziksel çekirdeklerin performans artışı için donanım tarafından **çoklu mantıksal işlemciler**e bölünmesidir. İşlemcinin aynı fiziksel alanı, aynı anda iki ayrı iş

parçacığı/talimat seti yürütür. Tek fiziksel çekirdek, İS'ye **iki ayrı işlemci gibi** görünür.

Windows'ta kontrol (WMI)

```
> wmic cpu get NumberOfCores, NumberOfLogicalProcessors
NumberOfCores  NumberOfLogicalProcessors
2                4
# 2 fiziksel çekirdek → 4 mantıksal işlemci = Hyperthreading aktif
```

Thread İptali (Cancellation)

- **Tanım:** Hedef thread'in, yürütmesi tamamlanmadan önce kasıtlı sonlandırılması. Senaryolar: çoklu thread veritabanı araması (biri bulunca diğerleri iptal), tarayıcıda 'Durdur' butonu.
- **Asenkron iptal (tehlikeli):** Hedef **anında** sonlandırılır. Risk: paylaşılan veri güncellenirken iptal edilirse kaynaklar **kilitli kalır, veri bozulur**.
- **Ertelenmiş iptal — Deferred (güvenli):** Hedef thread, **kendi periyodik bayraklarını (flag)** kontrol eder; kaynakları güvenli bırakabileceği uygun bir noktada kendini sonlandırır.

fork() ve exec() Çıkmazı (çok thread'li süreçte)

- Çok iş parçacıklı bir süreçte tek bir thread fork() çağırırsa: **fork'tan hemen sonra exec() çağırılacaksa → sadece çağırılan thread kopyalanır** (exec zaten tüm süreci değiştireceği için hepsini kopyalamak gereksiz yüküdür); **exec çağırılmayacaksa → süreçteki TÜM thread'ler kopyalanır** (bağımsız kopya, sorunsuz devam etmeli).

5.7 Amdahl Yasası ve Çok Çekirdek

Çok çekirdekli mimarinin kazancı, uygulamanın paralel kaynakları **etkin kullanabilmesine** bağlıdır.

Amdahl Yasası

Hızlanma = (1 işlemcide süre) / (N işlemcide süre) = $1 / ((1 - f) + f / N)$
 f : sonsuz paralelleştirilebilir kod oranı
 (1 - f): seri (paralelleştirilemeyen) kod oranı

- **Örnek:** Kodun yalnızca %10'u seri ise (f = 0,9), 8 işlemcili sistemde hızlanma sadece **4,7 kat** olur. Az miktardaki seri kod bile belirgin etki yapar.
- Ek yükler (iş dağıtımı, iletişim, önbellek tutarlılığı) nedeniyle performans bir noktada **zirve yapıp düşmeye** başlayabilir.
- Çok çekirdekten fayda sağlayan uygulamalar: çok iş parçacıklı yerel uygulamalar, çoklu işlem (multiprocess) uygulamaları (Oracle, SAP), Java/J2EE uygulamaları, çoklu örnekli (multi-instance) uygulamalar.
- **Örnek olay — Valve:** Source oyun motoru, Intel/AMD çok çekirdekli işlemcilerden yararlanmak için çoklu iş parçacığı kullanacak şekilde yeniden programlandı (hiyerarşik thread yapısı; üst düzey thread'ler gerektiğinde alt düzey thread'ler oluşturur).

6 | İşletim Sistemleri için C Programlama

6.1 C# ile C'nin Temel Farkları

Özellik	C#	C
Bellek yönetimi	Garbage Collector (otomatik)	Manuel — malloc / free
Tip güvenliği	Güçlü, derleme zamanı	Zayıf — pointer serbesttir
OOP	Tam nesne yönelimli	Yok — prosedürel
String	System.String nesnesi	char dizisi (null '\0' ile biter)
Derleme	.NET IL → JIT	Doğrudan makine koduna
Standart kütüphane	Zengin BCL	Minimalist libc
Hata yönetimi	Exception (try/catch)	Dönüş değerleri ve errno
Tip boyutu	int = 32 bit garantili	Platforma bağlı (sizeof kullan)

- Sözdizimi karşılıkları: #include ↔ using (başlık dosyaları .h); printf() ↔ Console.Write (format: %d int, %s string, %f float); typedef struct ↔ class/struct (new yok); erişim belirteçleri yok (static = dosya bazında erişim); main() int döndürür: 0 = başarı.

6.2 Pointer ve Bellek Yönetimi

Pointer + malloc/free

```
int x = 10;
int *p = &x;          // p, x'in ADRESİNİ tutar
printf("%d", *p);      // *p = dereference → 10
*p = 99;               // x artık 99

int *dizi = (int *) malloc(5 * sizeof(int)); // HEAP'ten yer iste
if (dizi == NULL) { perror("malloc"); return 1; } // MUTLAKA kontrol!
free(dizi);           // MUTLAKA serbest bırak → yoksa MEMORY LEAK
dizi = NULL;          // dangling pointer'ı engelle
```

- Stack (otomatik yerel değişkenler) vs Heap (malloc/free ile yönetilen alan). free() unutulursa **memory leak**.
- İS bağlantısı: çekirdek belleği de benzer yönetilir (kmalloc/kfree); stack overflow → **segfault** (İS sinyali: **SIGSEGV**).

6.3 Dosya G/Ç — Sistem Çağrıları

Her G/Ç işlemi aslında bir **kernel system call**'dur. İki seviye vardır:

Yüksek seviye (stdio.h)	Düşük seviye sistem çağrısı (unistd.h / fcntl.h)
FILE *f = fopen("log.txt", "w")	int fd = open("log.txt", O_WRONLY O_CREAT O_TRUNC, 0644)
fprintf(f, ...), fputs(...)	write(fd, msg, strlen(msg))
fgets(satir, n, f)	read(fd, buf, n)
fclose(f)	close(fd)

- Dosya tanımlayıcı (fd):** her açık dosya bir numardır. Sabitler: **0 = stdin**, **1 = stdout**, **2 = stderr**. Örn. write(1, buf, n) → standart çıktıya yazar.

6.4 Prosesler: fork() + exec() + waitpid()

fork + exec + waitpid

```

pid_t pid = fork(); // bu satırdan sonra İKİ proses çalışır
if (pid < 0) { perror("fork"); exit(1); }
else if (pid == 0) { // ÇOCUK: pid 0 döner
    printf("PID=%d, Ebeveyn=%d\n", getpid(), getppid());
    execlp("ls", "ls", "-l", NULL); // prosesi 'ls -l' ile değiştir
    perror("exec"); // buraya gelirse exec başarısızdır
    exit(1);
} else { // EBEVEYN: pid = çocuğun PID'i
    int durum;
    waitpid(pid, &durum, 0); // çocuğu bekle → ZOMBIE engelle!
    if (WIFEXITED(durum)) printf("Çıkış kodu: %d\n", WEXITSTATUS(durum));
}

```

- **fork()** → 0 dönerse çocuk, >0 dönerse ebeveyn. **exec()** → mevcut prosesi yeni programla değiştirir. **waitpid()** → zombie proses oluşumunu engeller.

6.5 Sinyaller (Signals)

Ctrl+C, segfault, timer... hepsi **sinyal** mekanizmasını kullanır: çekirdekten prosese bildirim.

Sinyal	No	Anlamı
SIGINT	2	Ctrl+C — kullanıcı kesmesi
SIGTERM	15	Kibarca sonlandırma (kill komutunun varsayılanı)
SIGKILL	9	Zorla öldür — YAKALANAMAZ!
SIGSEGV	11	Segmentation fault — hatalı bellek erişimi
SIGCHLD	17	Çocuk proses bitti
SIGALRM	14	Zamanlayıcı alarmı
SIGUSR1 / SIGUSR2	10 / 12	Kullanıcı tanımlı sinyaller

Sinyal yakalama

```

volatile int calis = 1;
void sinyal_handler(int sig) {
    if (sig == SIGINT) { printf("Ctrl+C yakalandı, temizlik...\n"); calis = 0; }
}
int main() {
    signal(SIGINT, sinyal_handler); // handler'ı kaydet
    while (calis) { printf("Çalışıyor...\n"); sleep(1); }
    printf("Proses düzgün sonlandı.\n");
}

```

6.6 IPC: Pipe ve Paylaşımlı Bellek

- **Pipe (boru):** tek yönlü veri akışı. `int fd[2]; pipe(fd);` → **fd[0] = okuma ucu, fd[1] = yazma ucu**. `fork()` sonrası: çocuk yazma ucunu kapatıp okur; ebeveyn okuma ucunu kapatıp yazar. Shell de pipe kullanır: `ls -la | grep .c`
- **Paylaşımlı bellek (POSIX) — en hızlı IPC:** `shm_open` (oluştur) → `ftruncate` (boyutlandır) → `mmap` (adres uzayına eşle) → doğrudan yazılır/okunur → `munmap` + `shm_unlink` (temizle).
- Diğer IPC mekanizmaları: Message Queue, Socket, FIFO (isimli pipe).

6.7 Thread ve Mutex — pthreads

pthread + mutex (mutex ile sonuç her zaman doğru)

```

long sayac = 0;
pthread_mutex_t kilit = PTHREAD_MUTEX_INITIALIZER;

void *thread_gorevi(void *arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&kilit);    // kritik bölge BAŞLA
        sayac++;                      // race condition YOK
        pthread_mutex_unlock(&kilit);  // kritik bölge BİTTİ
    }
    return NULL;
}

// main: pthread_create(&tid, NULL, thread_gorevi, &arg);
//      pthread_join(tid, NULL); // bitmesini bekle
// Derleme: gcc -o prog prog.c -lpthread

```

C# (Task / Thread)	pthread (C)
new Thread(fn)	pthread_create()
thread.Join()	pthread_join()
lock(obj) { }	pthread_mutex_lock / unlock
Semaphore	sem_t (semaphore.h)
CancellationToken	volatile flag + sinyal

- Pratik araçlar: gcc -Wall -Wextra (uyarılar), valgrind (bellek leak tespiti), strace (sistem çağrılarını izleme), man 2 fork.

7 | İşlem Senkronizasyonu

7.1 Temel Sorun: Yarış Durumu (Race Condition)

Bağımsız işlemler güvenle çalışabilir; ancak **paylaşılan kaynaklara erişen işbirlikçi işlemler (cooperative processes)** senkronize edilmezse **veri tutarsızlığı** yaşar.

- **Banka örneği:** Paylaşılan bakiye = 100\$. İşlem 1 bakiyeyi okur (100\$), 10\$ eklemeye hazırlanır → tam o anda **kesme (interrupt)** olur → İşlem 2 bakiyeyi okur (100\$), 10\$ çıkarır ve 90\$ yazar → İşlem 1 kaldığı yerden devam edip 110\$ yazar. **Sonuç:** Gerçek bakiye 100\$ olması gerekirken, sırasız çalışma nedeniyle 110\$ (veya 90\$) olur.
- Hatırlatma (Bölüm 1): counter++ bile atomik değildir (LOAD → INC → STORE); araya giren context switch tutarsızlık yaratır.

7.2 Kritik Bölge (Critical Section) Anatomisi ve Kuralları

Yarış durumlarını önlemek için paylaşılan kaynaklara erişen kod blokları (**kritik bölge**) sıkı kurallarla korunmalıdır.

- **Anatomi: Giriş Bölümü (Entry Section):** izin istenir → **Kritik Bölge (Critical Section):** paylaşılan veriye erişilir → **Çıkış Bölümü (Exit Section):** kilit/izin serbest bırakılır → **Kalan Bölge (Remainder Section):** bağımsız kod çalışır.

Bir çözümün sağlaması gereken 3 kural

Kural	Tanım
1. Karşılıklı Dışlama (Mutual Exclusion)	Aynı anda sadece TEK bir işlem kritik bölgede olabilir.
2. İlerleme (Progress)	Kritik bölge boşsa ve girmek isteyen işlemler varsa, seçim süresiz olarak ertelenemez.
3. Sınırlı Bekleme (Bounded Waiting)	Bir işlemin kritik bölgeye girme isteği sınırlı sürede karşılanmalı; açlık (starvation) yaşanmamalıdır.

7.3 Yazılım Katmanı Çözümleri: Dekker Algoritmasının Evrimi

İS veya donanım desteği olmadan, **salt yazılım mantığıyla** karşılıklı dışlama sağlama çabasıdır. Her sürümün kusuru, bir sonraki sürümü doğurur (sınavda 'hangi sürüm hangi kuralı ihlal eder' diye sorulabilir):

Sürüm	Yaklaşım	Kusur
Sürüm 1	Sadece sıra değişkeni (turn): işlemler sırayla girer (katı sıra değişimi / strict alternation).	Bir işlem sonsuza dek dururs a (veya yavaşsa) diğeri de giremez → İLERLEME (Progress) durur.
Sürüm 2	Girişten SONRA bayrak: işlem kritik bölgeye girince bayrağını kaldırır.	İki işlem de aynı anda girebilir → KARŞILIKLI DIŞLAMA ihlal edilir.
Sürüm 3	Girişten ÖNCE bayrak: önce niyet bildirilir, sonra beklenir.	İkisi de aynı anda bayrak kaldırırsa birbirini sonsuza dek bekler → KİLİTLENME (Deadlock).
Sürüm 4	Rastgele bekleme: çakışınca bayrağı indirip rastgele süre bekle.	Deadlock çözülür ama bir işlem sürekli kaybedebilir → SINIRLI BEKLEME ihlali (Açlık).
Sürüm 5 (Nihai Dekker)	Bayraklar (niyet) + sıra değişkeni (öncelik) birleşimi.	ÇÖZÜM: Üç şartı da sağlar.

- Derste verilen C# kodlarında: Sürüm 1 threadNumber ile katı sıra; Sürüm 2-3 thread1/thread2 bayrakları ile; Sürüm 5 favouredThread (öncelik) + wantsToEnter (niyet)

bayraklarını birleştirir. Bekleme, döngüde `Thread.Yield()` / boş döngü (spin-wait) ile yapılır.

7.4 Peterson ve Fırın (Bakery) Algoritmaları

- **Peterson (2 işlem):** Dekker'in daha zarif, modern versiyonu; `flag[i]` (niyet) ve `turn` (öncelik) değişkenlerini birleştirir.

Peterson – işlem i için

```
flag[i] = true;           // niyetimi bildir
turn = j;                // önceliği rakibe ver (kibarlık)
while (flag[j] && turn == j); // rakip istekli VE sıra ondaysa bekle
// ---- Kritik Bölge ----
flag[i] = false;         // çıktım, bayrağı indir
```

- **Fırın / Bakery (N işlem) — Leslie Lamport:** FCFS (ilk gelen ilk hizmet alır) kuralıyla çalışır. Her işleme **sözlüksel (lexicographical) sırayla** bilet verilir: (**bilet numarası, işlem ID**). En küçük bilete sahip işlem kritik bölgeye girer. Kilitlenme ve açlık yaşanmaz. **Dezavantaj:** yüksek karmaşıklık ve **meşgul bekleme (busy waiting)**.

7.5 Donanım Katmanı Çözümleri

Kesmeleri Devre Dışı Bırakmak (ilk deneme)

Bir işlem kritik bölgedeyken donanım kesmelerini kapatmak (Disable Interrupts → Kritik Bölge → Enable Interrupts). **Neden başarısız?**

- **1. Çoklu işlemcilerde yetersiz:** Kesmeleri bir CPU'da kapatmak, diğer CPU'lardaki işlemlerin paylaşılan veriye erişmesini engellemez.
- **2. Tehlikeli yetki:** Kullanıcı işlemlerine bu yetki verilirse, kötü veya hatalı bir işlem (kesmeleri geri açmayı unutursa) tüm sistemi çöktirebilir.
- **3. Performans:** Modern İS'lerde bu yöntem yalnızca **çekirdek (kernel) seviyesinde sınırlı durumlarda** kullanılır.

Atomik Donanım Komutları

- **ATOMİK** = araya hiçbir kesmenin giremeyeceği, tek ve bölünemez bir adım.
- **Test-and-Set (TAS):** 1) Eski değeri oku, 2) yeni değeri Dolu (true) yap, 3) eski değeri döndür — hepsi tek atomik adımda. Eski değer Boş (false) ise işlem kritik bölgeye girer; Dolu ise döngüde bekler.
- **Compare-and-Swap (CAS):** Bellekteki değer, beklenen değere **eşitse** onu yeni değerle değiştir. Gelişmiş kontrol sağlar; **lock-free (kilitsiz) senkronizasyon** algoritmalarının temelidir.

Spinlock (Dönme Kilidi)

TAS veya CAS üzerine inşa edilen temel kilit mekanizmasıdır. İşlem, kilit serbest kalana kadar işlemciyi meşgul ederek döngüde kalır (**Spin / Busy Wait:** 'Kilit açık mı? Hayır. Açık mı? Hayır...').

Avantajları	Dezavantajları
Çok çekirdekli sistemlerde KISA süreli beklemler için son derece hızlıdır.	Meşgul bekleme: işlemci döngülerini israf eder.
Bağlam değişimi (context switch) yükü yoktur.	Uzun süren kritik bölgeler ve tek işlemcili sistemler için uygun değildir.
	Sınırlı bekleme garantisi yoktur; adil değildir (açlık riski).

İS Katmanı: Dönmek Yerine Uyumak (Spin vs Sleep)

- Meşgul beklemenin israfını çözmek için İS, kilit alınamayan işlemleri **bloke edip uyku kuyruklarına (wait queue)** taşır (örn. Solaris'te Park/Unpark, Linux'ta **futex**).
- **Spin (donanım):** %100 CPU kullanımı, işlemci israfı; hızlı ama verimsiz. **Sleep (İS):** kilit alınamazsa işlem bloke edilir (park/sleep); kilit serbest kalınca İS işlemi uyandırır (unpark/wake); CPU diğer işlere ayrılır.

7.6 Mutex ve Semafor — Kavramsal Yanılgılar

En yaygın yanılgı: 'Mutex, sadece ikili (binary) bir semafordur.' **YANLIŞ!** Temel amaçları ve sahiplik mekanizmaları tamamen farklıdır:

Özellik	Mutex (Mutual Exclusion)	Semafor (Semaphore)
Doğası	Bir NESNEDİR (object).	Bir TAMSAYIDIR (integer).
Temel amaç	Kaynak erişimi (KİLİTLEME mekanizması).	SİNYALLEŞME (signaling) mekanizması.
Sahiplik	SIKI SAHİPLİK vardır: sadece kilidi alan işlem kilidi açabilir.	SAHİPLİK YOKTUR: bekleyen işlem farklı, sinyal gönderen işlem farklı olabilir.
Operasyonlar	Lock() ve Unlock()	Wait() / P() ve Signal() / V()
Kullanım alanı	Aynı anda tek işlemin girmesi gereken kritik bölgeler.	Görev koordinasyonu ve belirli durumların beklenmesi (örn. 'indirme bittiyse yazdır').

Semafor Türleri

- **İkili Semafor (Binary):** Değer sadece 0 veya 1 olabilir. Kritik bölgelerde karşılıklı dışlama için (Mutex'e benzer şekilde, ancak sahiplik kuralı olmadan) kullanılır. 0 = kilitli, 1 = açık.
- **Sayan Semafor (Counting):** Değer 0 ile N (kaynak sayısı) arasında değişir. Örnek: 5 yazıcıli sistem — her wait() değeri 1 azaltır; değer 0'ın altına düşerse işlem **bloke kuyruğuna** alınır; her signal() değeri 1 artırır ve kuyruktaki bir işlemi uyandırır.

► **SINAV NOTU:** Semaforun dezavantajı: yanlış kullanım (wait/signal sırasını karıştırma, unutma) **Kilitlenmeye (Deadlock)** veya **Öncelik Tersinmesine** yol açabilir.

7.7 Programlama Dili Katmanı: Monitörler (Monitors)

- Semaforların karmaşıklığını ve hata payını (wait/signal unutulması) çözmek için diller (örn. **Java synchronized**) monitörleri sunar.
- **Dış kalkan:** Otomatik karşılıklı dışlama — monitör nesnesine aynı anda **sadece bir işlem** girebilir.
- **İçerik:** Paylaşılan veri (korunan kaynaklar) + Metotlar (veriye yalnızca bu tanımlı prosedürlerle erişilir) + **Durum değişkenleri (condition variables):** işlemlerin beklemesini sağlar — komutlar: wait(), signal(), broadcast().
- **Önemli:** Monitörler işletim sistemine değil, **programlama dilinin derleyicisine** bağlıdır.

7.8 İleri Düzey Sorun: Öncelik Tersinmesi (Priority Inversion)

Tanım: Düşük öncelikli bir işlemin, kritik bir kilidi tutarak yüksek öncelikli bir işlemi süresiz beklemesine neden olduğu sistem arızasıdır.

- **Senaryo:** L (düşük öncelik) kilidi tutar → H (yüksek öncelik) gelir, kilit için bloke olur → M (orta öncelik) gelir ve L'yi **kesintiye uğratar (preempt)**, CPU'yu alır → Sonuç: H (en önemli görev), M (daha az önemli görev) yüzünden **süresiz durdurulur** (Sınırsız Tersinme / Unbounded Inversion).
- **Çözüm — Öncelik Mirası (Priority Inheritance Protocol):** İS, H kilit için L'yi beklemeye başlayınca **müdahale eder:** L'ye geçici olarak H'nin yüksek önceliğini verir (boost). Artık M,

L'yi kesintiye uğratamaz; L kritik bölgeyi hızla tamamlar ve kilidi bırakır; L'nin önceliği eskiye döner; H kilidi alıp çalışır.

► **SINAV NOTU: Tarihî olay:** Mars Pathfinder uzay aracında yaşanan ünlü sistem çökmesi (watchdog timer reboot) tam olarak bu problemdi ve Öncelik Mirası protokolü ile uzaktan çözülmüştür.

7.9 Senkronizasyon Hiyerarşisi (Nihai Sentez)

Katman	Araç	Ne zaman kullanılır?
Dil Katmanı	Monitörler (Monitors)	Nesne yönelimli programlamada, manuel wait/signal hatalarından kaçınmak ve veriyi otomatik korumak için. Yüksek seviyeli ve en güvenli yapıdır.
İşletim Sistemi Katmanı	Mutex ve Semaforlar	Mutex → kesin sahiplik ve uzun süreli kaynak kitleme. Semafor → görev sırası sinyalleşmesi ve havuzlanmış sınırlı kaynakların yönetimi.
Donanım Katmanı	Test-and-Set / CAS / Spinlock	Çok çekirdekli mimarilerde, çok kısa (mikrosaniye) sürecek kilitlenmelerde; context switch yükünden kaçınmak için.

8 | Klasik Senkronizasyon Problemleri ve Kodları

Öğrenilen tüm senkronizasyon araçları (kilitler, semaforlar, monitörler), bilgisayar bilimlerindeki klasik IPC sorunlarını çözmek için kullanılır. Bu bölümde **hocanın paylaştığı kod dosyaları satır satır** açıklanmıştır.

Klasik Problem	Temsil Ettiği Senaryo	Temel Tehlike	Tercih Edilen Çözüm
Üretici-Tüketici	İletişim kuyukları, veri akışı yönetimi	Taşıp veri kaybetme / boş okuma	Sayan semafor (buffer yönetimi) + mutex
Okuyucu-Yazar	Veritabanı ve dosya sistemi eşzamanlılığı	Yazar açlığı (starvation)	Çift mutex (okuma sayacı + yazma kilidi)
Yemek Yiyen Filozoflar	Çoklu kaynak gerektiren sistemler	Kilitlenme (deadlock)	Kaynak sıralama + monitörler
Uyuyan Berber	Sunucu (server) istek ve kuyruk yönetimi	Adil hizmet ve gereksiz meşguliyet	Semafor tabanlı bekleme kuyukları

8.1 Üretici-Tüketici (Producer-Consumer / Bounded Buffer)

- **Sorun:** Eşzamanlı işlemlerde paylaşılan sınırlı bir belleğin (bounded buffer) **taşmasını** veya **boşken okunmasını** önlemek. Tehlikeler: **Buffer Overflow** (tampon doluyken üretici eklemeye çalışır), **Buffer Underflow** (tampon boşken tüketici okumaya çalışır).
- **Standart çözüm — 3 semafor:** mutex = 1 (aynı anda erişimi engeller, ikili), empty = n (boş alan sayar, başlangıçta tampon boyutu), full = 0 (dolu alan sayar, başlangıçta 0).

Producer_Consumer.txt – hocanın kodu (açıklamalı)

```
semaphore mutex = 1; // karşılıklı dışlama (ikili)
semaphore full = 0; // başlangıçta dolu yuva yok
semaphore empty = n; // tampon boyutu kadar boş yuva

void Producer() {
    do {
        // bir öğe ÜRET
        wait(empty); // boş yuva var mı? (yoksa blokla; varsa empty--)
        wait(mutex); // kritik bölgeye gir
        // öğeyi tampona YERLEŞTİR
        signal(mutex); // kritik bölgeden çık
        signal(full); // dolu yuva sayısını artır (tüketiciyi uyandırabilir)
    } while (true);
}

void Consumer() {
    do {
        wait(full); // dolu yuva var mı? (yoksa blokla)
        wait(mutex); // kritik bölgeye gir
        // öğeyi tampondan AL
        signal(mutex); // kritik bölgeden çık
        signal(empty); // boş yuva sayısını artır (üreticiyi uyandırabilir)
    } while (true);
}
```

► **SINAV NOTU: Sıra hayati önemde:** Önce wait(empty)/wait(full), SONRA wait(mutex). Sıra ters çevrilirse (önce mutex alınıp sonra empty beklenirse) üretici mutex'i tutarken uyuyabilir ve tüketici asla mutex'i alamaz → **DEADLOCK**. Signal'lerin sırası ise kritik değildir.

8.2 Okuyucu-Yazar (Reader-Writer) Problemi

Veritabanı gibi paylaşılan bir yapıya **aynı anda birden fazla okuyucu** girebilirken, **yazar** işleminin **kesin bir karşılıklı dışlama** gerektirmesi durumudur.

Tasarım 1: Okuyucu Önceliği (Readers Preference) — Readers_Writers_1.txt

- **Etki:** Yeni okuyucular geldikçe yazarlar sürekli bekleyebilir → **YAZAR AÇLIĞI** riski.

Okuyucu önceliği – ilk okuyucu kilitler, son okuyucu açar

```
semaphore mutex = 1; // readcnt'yi güncellerken karşılıklı dışlama
semaphore wrt = 1; // paylaşılan VERİYE erişimi kontrol eder
int readcnt = 0; // kritik bölgedeki okuyucu sayısı

void Reader() {
    do {
        wait(mutex); // sayacı güncellemeden önce kilitler
        readcnt++;
        if (readcnt == 1)
            wait(wrt); // İLK okuyucu yazarları bloklar
        signal(mutex); // diğer okuyucular girebilsin

        // ---- Kritik Bölge (OKUMA) ---- (çok okuyucu aynı anda olabilir)

        wait(mutex);
        readcnt--;
        if (readcnt == 0)
            signal(wrt); // SON okuyucu yazarlara izin verir
        signal(mutex);
    } while (true);
}

void Writer() {
    do {
        wait(wrt); // kaynağı TEK BAŞINA kilitler
        // ---- Kritik Bölge (YAZMA) ----
        signal(wrt); // kaynağı bırak
    } while (true);
}
```

- **Mantık:** Okuyucular kendi aralarında çakışmaz; sadece **ilk giren** okuyucu yazarları kilitler (wait(wrt)) ve **son çıkan** okuyucu kilidi açar (signal(wrt)). mutex sadece readcnt sayacını korur.

Tasarım 2: Adil Çözüm / Yazar Açlığını Önleme — Readers_Writers_2.txt

- **Yazar Önceliği etkisi:** Yazar bekliyorsa yeni okuyucu alınmaz (okuyucu açlığı riski doğar); bu kod **serviceQueue** turnikesiyle geliş sıraya koyarak adaleti artırır.

Adil okuyucu-yazar – turnike (serviceQueue) + kapı (readTry)

```

semaphore resource      = 1; // asıl paylaşılan veri
semaphore readTry       = 1; // yazarların kapatabildiği 'okuyucu kapısı'
semaphore rmutex        = 1; // read_count'u korur
semaphore wmutex        = 1; // write_count'u korur
semaphore serviceQueue  = 1; // geliş sırasını koruyan TURNİKE
int read_count = 0, write_count = 0;

void reader() {
    wait(serviceQueue); // global sıraya gir (yazarın önüne geçemezsin)
    wait(readTry);      // yazar kapıyı kapattıysa burada beklersin
    signal(serviceQueue); // sıradaki gelsin
    wait(rmutex);
    read_count++;
    if (read_count == 1) wait(resource); // ilk okuyucu kaynağı kilitler
    signal(rmutex);
    signal(readTry);      // kapıyı bırak (sıradaki yazar alabilir)
    // ---- Kritik Bölge (oku) ----
    wait(rmutex);
    read_count--;
    if (read_count == 0) signal(resource); // son okuyucu kaynağı bırakır
    signal(rmutex);
}

void writer() {
    wait(wmutex);
    write_count++;
    if (write_count == 1) wait(readTry); // İLK bekleyen yazar KAPIYI KAPATIR
    signal(wmutex);                  // → yeni okuyucu giremez
    wait(serviceQueue); // okuyucularla AYNI sıraya gir
    wait(resource);      // özel (exclusive) erişim
    signal(serviceQueue);
    // ---- Kritik Bölge (yaz) ----
    signal(resource);
    wait(wmutex);
    write_count--;
    if (write_count == 0) signal(readTry); // son yazar kapıyı yeniden açar
    signal(wmutex);
}

```

- **Kilit fikirler:** 1) serviceQueue herkesin (okuyucu+yazar) geliş sırasını korur; kimse sıra atlayamaz. 2) İlk yazar readTry kapısını kapatınca **yeni okuyucular birikmez**; mevcut okuyucular bitince yazar girer → yazar açlığı önlenir.

8.3 Yemek Yiyen Filozoflar (Dining Philosophers)

Kaynakların işlemlere dağıtımı sırasında **Kilitlenme (Deadlock)** ve **Açlık (Starvation)** tehlikelerini modelleyen klasik **Dijkstra** problemidir. **Senaryo:** 5 filozof düşünür veya yemek yer; yemek için sağ ve sol olmak üzere **İki çubuğa** ihtiyaç vardır. **Tehlike:** Her filozof aynı anda sadece sol çubuğunu alırsa, hepsi sonsuza dek sağ çubuğun boşalmasını bekler → sistem tamamen durur (**döngüsel bekleme**).

Çözüm 1: Masayı Sınırlamak — Dining_Philosophers_1.txt

Filozoflar 1 – odaya en fazla 4 filozof

```

semaphore room = 4; // SAYAN semafor: masada en çok 4 kişi
semaphore chopstick[5] = {1,1,1,1,1}; // her çubuk bir İKİLİ semafor

void Philosopher(int i) {
    while (true) {
        /* düşün */
        wait(room); // masada yer var mı? (5. filozof bekler)
        wait(chopstick[i]); // kendi (sağ) çubuğunu al
        wait(chopstick[(i+1)%5]); // soldaki çubuğu al
        /* ye */ // kritik bölge
        signal(chopstick[i]); // çubukları bırak
        signal(chopstick[(i+1)%5]);
        signal(room); // masadan kalk
    }
}

```

- **Neden çalışır?** 5 çubuk, 4 filozof: **en az bir filozof iki çubuğu da alabilir** → döngüsel bekleme koşulu kırılır, deadlock imkânsızlaşır. (Bölüm 9'daki formülle uyumlu: $m \geq n(K-1)+1$ → $5 \geq 4 \cdot (2-1) + 1 = 5$ ✓)

Çözüm 2: Asimetri / Sıralamayı Kırmak — Dining_Philosophers_2.txt**Filozoflar 2 – tek/çift asimetrisi**

```

semaphore chopstick[5] = {1,1,1,1,1};

void Philosopher(int i) {
    while (true) {
        /* düşün */
        if (i % 2 == 1) { // TEK numaralı filozof:
            wait(chopstick[i]); // önce SAĞ (kendi) çubuğu
            wait(chopstick[(i+1)%5]); // sonra SOL çubuğu
        } else { // ÇİFT numaralı filozof:
            wait(chopstick[(i+1)%5]); // önce SOL çubuğu
            wait(chopstick[i]); // sonra SAĞ çubuğu
        }
        /* ye */ // kritik bölge
        signal(chopstick[i]);
        signal(chopstick[(i+1)%5]);
    }
}

```

- **Neden çalışır?** Herkes aynı yönde (önce sol) uzanmadığı için 'herkes bir çubuk alıp bekler' kalıbı (döngü) oluşmaz; komşu tek-çift filozoflar aynı çubuğa **farklı öncelik**le uzanır → biri mutlaka iki çubuğu alır. Bu, deadlock'un **Circular Wait** koşulunu kaynak sıralamasıyla kırmaktır.

8.4 Uyuyan Berber (Sleeping Barber)

Bir hizmet sisteminde isteklerin bekletilmesi, kuyruğa alınması ve adil hizmet almasının senkronizasyonudur. **Akış kuralları:** Müşteri yoksa berber **uyur** (CPU tasarrufu). Müşteri gelirse berberi **uyandırır** (sinyal / wakeup). Berber meşgulse ve boş koltuk varsa müşteri **oturup bekler** (kuyruk / wait). Koltuk yoksa müşteri dükkândan **ayrılır** (işlem düşürme).

Sleeping_Barber.txt – açıklama

```

Semaphore customers = 0; // bekleyen müşteri sayısı (berberi uyandırır)
Semaphore barber    = 0; // berberin hazır olduğunu bildirir
Mutex    mutex      = 1; // freeSeats'i korur
int freeSeats = N;     // bekleme odasındaki boş koltuk sayısı

Barber {
    while (true) {
        wait(customers); // müşteri yoksa UYU (0'da bloklanır)
        wait(mutex);
        freeSeats++;      // bir bekleme koltuğu boşaldı (müşteri kalktı)
        signal(barber);   // 'berber hazır' sinyali → müşteri koltuğa geçer
        signal(mutex);
        // saç kes
    }
}
Customer {
    while (true) {
        wait(mutex);
        if (freeSeats > 0) {
            freeSeats--; // koltuğa otur
            signal(customers); // berberi uyandır / kuyruğa kaydol
            signal(mutex);
            wait(barber);      // berber hazır olana kadar bekle
            // tıraş ol
        } else {
            signal(mutex); // koltuk yoksa dükkândan ayrıl
        }
    }
}

```

- **Çözümün özü:** Kuyruk sayısını (customers), koltuk durumunu (freeSeats) ve berberin aktifliğini (barber) senkronize etmek için semaforlar kullanılır. customers=0 sayesinde iş yokken berber CPU harcamaz (busy-wait yerine uyur).

► **SINAV NOTU:** Kod okuma sorusu gelirse ezber kalıbı: **wait() = azalt, 0'ın altına incekse blokla; signal() = artır, bekleyen varsa uyandır.** Başlangıç değerleri her şeyi anlatır: mutex=1 (kilit), sayaç semaforları = kaynak adedi, olay semaforları = 0 (henüz olay yok).

9 | Deadlock (Kilitlenme)

9.1 Tanım ve Gerekli Koşullar

Deadlock: İki veya daha fazla süreç, **birbirinin tuttuğu kaynakları beklediği** için **sonsuz** kadar **bloke** olduğunda meydana gelir. Klasik örnek: P1, R1'i tutuyor ve R2'yi istiyor; P2, R2'yi tutuyor ve R1'i istiyor → her iki süreç de sonsuza kadar bekler.

Coffman Koşulları — 4 Gerekli Koşul

Koşul	Açıklama
1. Mutual Exclusion (Karşılıklı Dışlama)	Kaynak aynı anda yalnızca bir süreç tarafından kullanılabilir; paylaşılamaz.
2. Hold and Wait (Tut ve Bekle)	Bir süreç en az bir kaynak tutarken başka kaynaklar için bekler.
3. No Preemption (Zorla Almama)	Bir sürecin tuttuğu kaynak zorla alınamaz; süreç kendi bırakmalıdır.
4. Circular Wait (Döngüsel Bekleme)	Süreçler halka şeklinde birbirinin kaynağını bekler: P1→P2→P3→P1.

► **SINAV NOTU:** Deadlock oluşması için 4 koşulun **TÜMÜ aynı anda** sağlanmalıdır. Önleme stratejilerinin mantığı: bu koşullardan **en az birini** ortadan kaldırmak.

9.2 Deadlock Yönetimi — 4 Temel Yaklaşım

Yöntem	Deadlock'a izin verir mi?	Overhead	Not
1. Önleme (Prevention)	Hayır	Düşük-Orta	4 koşuldan en az biri ortadan kaldırılır; kısıtlayıcı kurallar gerektirir.
2. Kaçınma (Avoidance)	Hayır	Yüksek	Tahsis öncesi sistem 'güvenli durumda mı' kontrol edilir (Banker Alg.); önceden bilgi gerektirir.
3. Tespit ve Kurtarma	Evet	Orta	Oluşmasına izin verilir; periyodik kontrolle tespit edilip çözülür; veri kaybı riski.
4. Görmezden Gelme (Ignorance)	Evet	Çok düşük	Deadlock çok nadirse sistem yeniden başlatılır (Devekuşu Algoritması).

9.3 Deadlock Önleme: Koşulları Yok Etme

- **Mutual Exclusion'ı kaldır:** Paylaşılabılır kaynaklar (salt-okunur dosyalar vb.) aynı anda birden çok sürece verilebilir. Paylaşılamaz kaynaklar için uygulanamaz.
- **Hold & Wait'i kaldır:** ① Tüm kaynaklar çalışmaya başlamadan önce talep edilir, VEYA ② yeni kaynak istemeden önce eldeki tüm kaynaklar bırakılır.
- **No Preemption'ı kaldır:** İstenen kaynak müsait değilse süreç, elindeki tüm kaynakları bırakır ve tümü hazır olana kadar bekler.
- **Circular Wait'i kaldır:** Kaynaklara benzersiz numaralar verilir; süreçler yalnızca **artan sırayla** kaynak talep edebilir → döngü oluşması imkânsızlaşır. (Filozoflar 2 çözümünün genel hali!)

9.4 Banker Algoritması (Kaçınma)

Temel Veri Yapıları

Yapı	Tanım
Available (Müsait)	Her kaynak türünden mevcut (boşta) örnek (instance) sayısını tutan 1B dizi.
Max (Maksimum)	Her sürecin en fazla talep edebileceği kaynak sayısını tutan 2B matris.
Allocation (Tahsis)	Hâlihazırda her sürece tahsis edilmiş kaynak sayısı (2B matris).
Need (İhtiyaç)	$Need = Max - Allocation$. Sürecin hâlâ ihtiyaç duyabileceği kaynak miktarı.

- **Güvenli Durum (Safe State):** Tüm süreçlerin sırayla tamamlanabileceği **en az bir sıralama** mevcuttur → deadlock riski yok. **Güvensiz Durum:** Tüm süreçlerin biteceği garanti edilemez → **potansiyel** deadlock riski (kesin deadlock demek değildir).

Güvenlik Algoritması Adımları

- **1)** $Work = Available$; tüm süreçler için $Finish[i] = false$.
- **2)** $Finish[i] = false$ VE $Need[i] \leq Work$ koşulunu sağlayan bir P_i bul.
- **3)** Bulunursa: $Work = Work + Allocation[i]$; $Finish[i] = true$; Adım 2'ye dön. (Süreç bitti, kaynaklarını iade etti.)
- **4)** Tüm $Finish[i] = true$ ise sistem **güvenli**; değilse **güvensiz**.

Slayttaki Örnek (3 kaynak türü: A, B, C — Available: A=3, B=3, C=2)

Süreç	Allocation (A B C)	Max (A B C)	Need = Max-Alloc (A B C)
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Adım adım çözüm: $Work=(3,3,2)$. $Need(P1)=(1,2,2) \leq Work \checkmark \rightarrow P1$ biter, $Work=(3,3,2)+(2,0,0)=(5,3,2)$. $Need(P3)=(0,1,1) \leq Work \checkmark \rightarrow Work=(5,3,2)+(2,1,1)=(7,4,3)$. $Need(P4)=(4,3,1) \leq Work \checkmark \rightarrow Work=(7,4,3)+(0,0,2)=(7,4,5)$. $Need(P2)=(6,0,0) \leq Work \checkmark \rightarrow Work=(10,4,7)$. $Need(P0)=(7,4,3) \leq Work \checkmark \rightarrow Work=(10,5,7)$. Tümü bitti → sistem **güvenli**; güvenli sıra: **P1 → P3 → P4 → P2 → P0**.

9.5 Deadlock Tespiti ve Kurtarma

Tespit

- **Tek instance kaynaklar: Resource Allocation Graph (RAG)** kullanılır. Grafikte **döngü (cycle) varsa deadlock KESİNDİR**. RAG'dan yalnızca süreç düğümlerini içeren **Wait-for Graph** türetilir ve periyodik olarak döngü aranır.
- **Çoklu instance kaynaklar:** Banker'a benzer bir tespit algoritması (Work, Allocation, Request matrisleri). Burada **döngü gerekli ama yeterli koşul DEĞİLDİR**. Sonuçta $Finish[i]=false$ kalan tüm süreçler deadlock içindedir.

RAG Bileşenleri

- **Süreç düğümü:** daire (○). **Kaynak düğümü:** kare (□); çok instance'lı kaynak, içinde noktalar olan kare.
- **Tahsis kenarı (Assign Edge) R→P:** kaynak sürece tahsis edilmiş. **İstek kenarı (Request Edge) P→R:** süreç o kaynağı bekliyor.

Kaynak Türü	Grafik Durumu	Sonuç
Tek instance	Döngü VAR	Deadlock KESİNDİR

Kaynak Türü	Grafik Durumu	Sonuç
Tek instance	Döngü YOK	Deadlock yoktur
Çoklu instance	Döngü VAR	Deadlock OLMAYABİLİR (yeterli koşul değil)
Çoklu instance	Döngü YOK	Deadlock yoktur

Kurtarma

- **Süreç sonlandırma:** ① Tüm deadlock'lu süreçleri sonlandır (kesin çalışır ama büyük veri kaybı riski); ② Birer birer sonlandır (her sonlandırmadan sonra tespit algoritması tekrar çalıştırılır; daha az kayıp).
- **Kaynak Preemption (zorla alma): Kurban seçimi** (maliyet hesabıyla hangi sürecin kaynakları alınacak), **Rollback** (süreç güvenli bir önceki duruma döndürülür), **Açlık önleme** (aynı süreç hep kurban seçilmemeli; bekleme sayısı sınırlanır).

9.6 Açlık (Starvation) ve Livelock

- **Açlık:** Düşük öncelikli süreçler, yüksek öncelikliler sürekli CPU/kaynak aldığı için belirsiz süre bloke kalır. Nedenler: öncelik zamanlaması, kaynakların hep öncelikliye gitmesi, aşırı preemption. **Çözüm → Aging (Yaşlandırma):** bekleme süresi arttıkça sürecin önceliği de artırılır.
- **Livelock:** Süreçler **bloke değildir** ama çakışmayı önlemek için sürekli durum değiştirir ve **hiçbir ilerleme olmaz** (aşırı kibarlık: iki süreç sürekli birbirine yol verir; busy-wait döngüleri; çakışan yanıt stratejileri). **Çözümler:** rastgele bekleme, exponential backoff, merkezi koordinatör.

9.7 Kaynak Tahsis Teknikleri ve Deadlock-Free Koşulu

- **Bölümleme (Partitioning):** İS, kaynakları sabit bölümlere ayırır; her bölüm bir programa atanır. + Kolay uygulama, düşük yönetim maliyeti; – esneklik eksikliği, kaynak israfı (atıl kaynak).
- **Havuz Tabanlı (Pool-Based):** Kaynaklar ortak havuzda tutulur; talep edince dinamik atanır. + Verimli ve esnek; – daha karmaşık yönetim, deadlock riski artabilir.

Deadlock-Free (Kilitlenmesiz) Koşul

Kilitlenmesiz çalışma koşulu: $m \geq n \times (K - 1) + 1$

m: toplam kaynak sayısı | n: süreç sayısı | K: bir sürecin ihtiyaç duyduğu MAKSİMUM kaynak

Örnek 1: 3 süreç, her biri en çok 2 kaynak → $m \geq 3 \times (2-1) + 1 = 4$ → en az 4 kaynak gerekir.

Örnek 2: 5 süreç, her biri en çok 3 kaynak → $m \geq 5 \times (3-1) + 1 = 11$ → en az 11 kaynak gerekir.

✓ **BÖLÜM ÖZETİ:** Deadlock = 4 koşulun (ME · H&W · NP · CW) aynı anda gerçekleşmesi. Yönetim sırası: Önleme → Kaçınma (Banker) → Tespit & Kurtarma → Görmezden gelme. RAG, tek-instance tespitin temel aracıdır. Açlık ve livelock deadlock'a benzer ama farklı çözümler gerektirir.

10 | Bellek Yönetimi ve Adresleme

10.1 Bellek Yönetimi Nedir?

- Bilgisayar belleğinin denetim altında tutulması ve düzenlenmesi sürecidir; çalışan programlara bloklar halinde bellek tahsis eder. İS'nin en kritik görevlerinden biridir.
- Aynı anda **birden fazla sürecin bellekte bulunmasını** sağlar; süreçleri **yetkisiz erişimden korur; takas (swapping)** ve **sanal bellek** kullanımını etkin kılar.

10.2 Bellek Tahsis Teknikleri

A) Bitişik Tahsis (Contiguous) — her işlem TEK sürekli blok alır

- 1. Sabit Bölümleme (Fixed Partitioning):** Bellek, sabit sayıda ve sabit boyutlu bölümlere ayrılır; her bölüme tek işlem yerleşir; İS bölümleri **bölüm tablosuyla** izler. + Basit; – bellek verimliliği düşük: **DAHİLİ parçalanma**; multitasking için elverişsiz.
- 2. Değişken Bölümleme (Variable Partitioning):** Bölüm boyutları işleme göre dinamik belirlenir. Dahili parçalanmayı azaltır ama **HARİCİ parçalanmaya** yol açar.
- Yerleştirme algoritmaları:** **First Fit** (ilk yeterli boş blok), **Best Fit** (en küçük uygun blok), **Worst Fit** (en büyük blok), **Next Fit** (son tahsisin kaldığı yerden aramaya devam).

B) Bitişik Olmayan Tahsis (Non-Contiguous) — işlem parçalara bölünür

- Sayfalama (Paging):** İşlem adres uzayı **eşit boyutlu sayfalara**, fiziksel bellek aynı boyutlu **çerçevelere (frame)** bölünür; **sayfa tablosu** sayfaları çerçevelere eşler. Harici parçalanmayı **ortadan kaldırır**; küçük miktarda dahili parçalanma olabilir.
- Segmentasyon:** İşlem **mantıksal bölümlere** (kod, veri, yığın) ayrılır; segment boyutları **değişkendir**; kullanıcıya daha anlamlı bir bellek görünümü sunar.
- Adres çevirisi için **MMU (Memory Management Unit)** donanım desteği gerekir; sanal bellekli modern İS'lerin temel yaklaşımıdır.

10.3 Sanal Bellek, Demand Paging, Sayfa Değiştirme

- Sanal bellek:** Programın fiziksel RAM'den büyük olmasına olanak tanır; disk, ana belleğin uzantısı gibi kullanılır.
- Talep üzerine sayfalama (Demand Paging):** Yalnızca **gerçekten ihtiyaç duyulan** sayfalar belleğe yüklenir.
- Sayfa takası (Page Swapping):** RAM yetersiz kalınca bazı sayfalar geçici olarak diske taşınır.

Sayfa Değiştirme Algoritması	Kuralı
FIFO (First-In First-Out)	Belleğe ilk giren (en eski) sayfa ilk çıkarılır.
LRU (Least Recently Used)	En uzun süredir KULLANILMAYAN sayfa çıkarılır.
Optimal	Gelecekte en geç kullanılacak sayfa çıkarılır (teorik; geleceği bilmek gerekir → uygulanamaz, kıyas ölçütüdür).
LFU (Least Frequently Used)	En az SIKLIKTA kullanılan sayfa çıkarılır.

10.4 Parçalanma (Fragmentation) ve Thrashing

- Dahili Parçalanma:** Tahsis edilen blok ihtiyaçtan büyüktür; **blok içindeki** alan boşa gider (sabit bölümleme, sayfalamanın son sayfası).
- Harici Parçalanma:** Toplam serbest alan yeterli olsa da **büyük bir bitişik blok bulunamaz** (değişken bölümleme, sürekli tahsis).

- **Çözüm — Sıkıştırma (Compaction):** Dağınık boş alanlar birleştirilir.
- **Thrashing:** Sistem, işleri yürütmek yerine zamanının büyük bölümünü **sayfa takası** yaparak geçirir; CPU kullanımı dramatik düşer. Çok fazla işlem aynı anda bellekteyse oluşur.
- **Önleme — Working Set modeli:** her sürecin çalışma kümesi (en az kaç sayfa gerektiği) hesaplanır; belleğin toplam working set'i karşılayıp karşılamadığına bakılır.

10.5 Mantıksal ve Fiziksel Adres, MMU

- **Mantıksal (Sanal) Adres:** CPU'nun program çalışırken ürettiği adres; işlemin kendi perspektifidir, fiziksel olarak var olmaz. **Mantıksal adres uzayı** = işlemin üretebileceği tüm mantıksal adresler kümesi.
- **Fiziksel (Gerçek) Adres:** RAM'de veri/komutların depolandığı gerçek konum; kullanıcı tarafından doğrudan görülemez.
- **MMU:** CPU ile bellek arasındaki donanım birimi; mantıksal adresleri **sayfa tablosu** aracılığıyla fiziksel adreslere **şeffaf biçimde** çevirir. Akış: **CPU → Mantıksal Adres → MMU → Fiziksel Adres → RAM**.

Özellik	Mantıksal Adres	Fiziksel Adres
Üreten	Program çalışırken CPU	MMU
Fiziksel varlık	Yok (sanal)	RAM'deki gerçek konum
Kullanıcı erişimi	Görülebilir ve erişilebilir	Doğrudan görüntülenemez
Değişkenlik	Çalışma sırasında değişebilir	Atandıktan sonra genellikle sabit
Diğer adı	Sanal adres	Gerçek adres

11 | Sanal Bellek (Derinlemesine)

11.1 Neden Sanal Bellek?

Sanallaştırmazsanız bellek sisteminin **üç temel sorunu** vardır: 1) Sistemde tüm programların gereksinimini karşılayacak kadar büyük bellek olmayabilir. 2) Programcı, programın çalışacağı sistemdeki **bellek boyutunu bilemez**. 3) Programcı, kendi programının yanında **hangi programların çalışacağını bilemez**.

- Çözüm: Her programın kendi **sanal adres uzayı** vardır; programdaki adres referansları sanal adrestir; her programa atanmış bir fiziksel bellek bölümü bulunur.
- **Kazanımlar:** ① Programcı, hangi fiziksel adreslerin atandığını bilmeden program yazar (iki program aynı sanal 0x0 adresini kullanabilir). ② Programların birbirinin belleğine doğrudan erişimi yoktur → **güvenlik**. ③ Ana bellek ↔ ikincil bellek veri akışıyla **'bellek sınırsızmış' yanılması** sağlanır.

11.2 Sayfalar ve Adres Dönüşümü

- İkincil bellek ile ana bellek arasındaki veri akışı **sayfalar** (sabit boyutlu bellek blokları) aracılığıyla yapılır.
- **Adres dönüşümü (page translation):** Sanal Adres = **[Sanal Sayfa Numarası | Sayfa Ekleme (offset)]** → dönüşüm → Fiziksel Adres = **[Çerçeve (Fiziksel Sayfa) Numarası | aynı offset]**. Yalnızca sayfa numarası çevrilir; offset değişmez.

11.3 Sayfa Tablosu (Page Table)

- Sanal ↔ fiziksel sayfa eşleşmeleri, **bellekte** tutulan **sayfa tablosu** yapısında saklanır. Tablonun adresi bir **sayfa tablosu yazmacında** tutulur; erişimde önce bu yazmaçla tabloya gidilir.

Sayfa Tablosu Girişi Alanı	Açıklama
Çerçeve No (Fiziksel Sayfa No)	Fiziksel çerçeve adresi.
Valid bit (G)	Sayfa şu anda ana bellekte mi?
Dirty bit	Sayfa belleğe alındıktan sonra değiştirildi mi? (Değiştirilirse atılmadan önce diske yazılmalı.)
Protection	R/W/X erişim izinleri.
Reference bit	Sayfaya son zamanlarda erişildi mi? (LRU benzeri algoritmalara yardım eder.)

Belleğe Erişim Örneği (Id a4, -24(s0))

- 1) Sanal sayfa numarası ile sayfa tablosuna erişilir. 2) Karşılık gelen fiziksel sayfa numarası döner. 3) [Fiziksel sayfa no + offset] ile belleğe erişilir. 4) İstenen veri okunur.

► **SINAV NOTU:** Bir bellek erişimi için **İKİ erişim** yapılır (önce tabloya, sonra veriye) → **sanal bellek sistemi hızlandırmaz**; kendi başına yavaşlatır bile. Bu yüzden TLB gereklidir.

11.4 TLB — Etkin Sayfalar Ön Belleği (ESÖ)

- Dönüşümler için **işlemcide tutulan ön bellek**: Translation Lookaside Buffer. Dönüşümde **önce TLB kontrol edilir**.
- **TLB hit:** dönüşüm bulunur → doğrudan fiziksel adresle belleğe erişilir (tek erişim).

- **TLB miss:** sayfa tablosuna gidilir, dönüşüm alınır (ve TLB'ye yazılır), sonra fiziksel erişim yapılır.
- Bazı şartlar sağlandığında **TLB ve veri önbelleğine aynı anda (paralel) erişilebilir** → ek hızlanma.

11.5 Sayfa Hatası (Page Fault)

- **Tanım:** Sanal sayfaya karşılık gelen fiziksel sayfanın **ana bellekte bulunamaması**. İki nedeni olabilir: ① fiziksel sayfa **ikincil bellektedir**; ② fiziksel sayfa **henüz atanmamıştır**.
- Sayfa hatasında denetim **işletim sistemine** geçer. İS sayfayı ikincil bellekte arar. Hangi sayfaların diske alınacağı önceden bilinemediği için diskte bu sayfalara ayrılmış özel bölge vardır: **takas alanı (swap space)**; sayfaların burada nerede durduğunun kaydı da tutulur (sayfa tablosunda ya da ayrı bir tabloda).

Sayfa Hatası İşlem Adımları (sınavda sıralama sorulabilir)

- 1) Sayfa hatası **kesmesi** oluşur.
- 2) CPU; PC ve kritik durum bilgilerini otomatik saklar, **çekirdek yığının**a kaydeder.
- 3a) Boş fiziksel sayfa varsa: İS boş çerçeveye sanal sayfayı eşleştirir, eşleşmeyi sayfa tablosuna yazar.
- 3b1) Boş çerçeve yoksa: bir sayfa **kurban** seçilir (FIFO / LRU / LFU; Optimal teoriktir).
- 3b2) Kurban **kirli (dirty)** ise önce ikincil belleğe yazılır; çerçevesi yeni sayfaya tahsis edilir.
- 4) Gerekli sayfa diskten ilgili çerçeveye **yüklenir**.
- 5) **Sayfa tablosu ve TLB güncellenir**.
- 6) Hataya neden olan buyruk, çekirdek yığınınındaki PC sayesinde **tekrar getirilir**; program kaldığı yerden devam eder.

Yazma İşlemi

- Önbellek-ana bellek arasında iki yöntem vardır: doğrudan yazma (write-through) ve sonradan yazma (write-back). **Sanal bellekte** ikincil bellek-ana bellek gecikmesi ÇOK büyük olduğundan **her zaman SONRADAN YAZMA (write-back)** uygulanır.

12 | Dosya Sistemleri ve Disk Yönetimi

12.1 Dosya Sistemi Nedir?

- İS'nin depolama aygıtlarındaki veriyi **organize etme ve yönetme** yöntemidir; kullanıcı uygulamaları ile fiziksel donanım arasında **köprü** görevi görür. Kullanıcı dosyalara diskten bağımsız olarak **isim ve izin** üzerinden erişir.
- Katmanlı yapı:** Kullanıcı Uygulaması → Mantıksal Dosya Sistemi → **VFS (Sanal Dosya Sistemi)** → Fiziksel Dosya Sistemi → Disk Bölümleri. **CRUD** işlemleri: Create, Read, Update, Delete. Dosya adı = **Ad + Uzantı** (rapor.docx).

Dosya Sistemi	Özellik
FAT	Eski Windows; basit yapı, geniş uyumluluk.
NTFS	Modern Windows; izin, sıkıştırma, şifreleme destekler.
ext2/3/4	Linux/Unix'te yaygın; ext4 journaling (günlükleme) destekler.
HFS / APFS	Apple macOS/iOS; APFS modern ve hızlıdır.
VFS	Farklı dosya sistemlerini TEK arayüz altında birleştirir.
FAT32 vs exFAT	exFAT büyük dosyaları (>4GB) destekler; USB'lerde yaygın.

- Temel kavramlar:** **Dosya** (diskte isimle depolanan veri birimi); **Dizin** (dosya ve alt dizin listesini içeren özel dosya); **Meta veri** (ad, boyut, tarih, izinler); **Blok** (diskin en küçük tahsis birimi, 512 B–4 KB); **Bit vektörü/Bitmap** (her blok için 0=boş, 1=dolu); **Boundary Tag** (her blok kendi boyut/durumunu işaretler).

12.2 Dizin Yapıları

- Tek Seviyeli:** Tüm dosyalar tek dizinde; farklı kullanıcılar aynı adlı dosya oluşturamaz; küçük sistemlerde basit ama yetersiz.
- İki Seviyeli:** Her kullanıcıya ayrı izin; aynı adda farklı dosyalar mümkün; kullanıcılar arası paylaşım zordur.
- Ağaç Yapılı:** En yaygın (Unix/Windows); verimli arama ve mantıksal gruplama.
- Yol adları:** **Mutlak yol** kökten başlar (/usr/local/bin/python); **görelî yol** geçerli dizinden başlar (./dosyalar/veri.csv). Windows'ta sürücü harfi (C:\...). Özel semboller: . geçerli dizin, .. üst dizin, ~ kullanıcının ev dizini.
- Dizin tutmanın faydaları:** verimlilik (hızlı erişim), isimlendirme (çakışmasız aynı isimler), gruplama (mantıksal kategoriler).

12.3 Dosya Tahsis Yöntemleri

Yöntem	Tanım	Avantaj	Dezavantaj
Sürekli (Contiguous)	Dosya art arda bitişik bloklarda; dizinde başlangıç bloğu + uzunluk tutulur.	Rastgele erişim ÇOK hızlı; sıralı okuma mükemmel (az kafa hareketi); basit.	HARİCİ parçalanma; dosya boyutu önceden bilinmeli; büyüyünce yer bulunamayabilir. Kullanım: CD/DVD, gömülü sistemler.
Bağlı (Linked)	Her blok bir SONRAKİ bloğun adresini taşır; dizinde ilk ve son blok adresi.	Harici parçalanma yok; dosyalar büyüyebilir; dizin yapısı basit.	Rastgele erişim YAVAŞ (n. blok için n–1 blok gezilir); pointer için ekstra alan; zincir koparsa dosya kaybolur. FAT = bağlı tahsisin özel çeşidi (bağlantılar ayrı tabloda).

Yöntem	Tanım	Avantaj	Dezavantaj
İndeksli (Indexed)	Her dosya için bir İNDEKS BLOĞU; tüm blok adresleri buraya yazılır; dizin indeks bloğunu gösterir.	Rastgele erişim etkili (doğrudan i. adres); harici parçalanma yok; büyük dosyalar desteklenir. En esnek ve yaygın yöntem.	İndeks bloğu için ek alan; küçük dosyalarda israf (dahili parçalanma).

- **Unix/Linux inode yapısı: Doğrudan bloklar** (küçük dosyalar) + **tek dolaylı** (orta) + **çift dolaylı** (büyük) + **üçlü dolaylı** (çok büyük dosyalar) işaretçileri.

Dosyaya Erişim Yöntemleri

- **Sıralı (Sequential):** baştan sona sırayla; en basit. Örn: cp komutu kaynak dosyayı sıralı okur, hedefe sıralı yazar; metin/ses/video akışı.
- **Doğrudan/Rastgele (Direct/Random):** herhangi bir konuma blok numarasıyla atlayarak erişim. Örn: veritabanı, tablodaki 5000. kaydı doğrudan okur.
- **İndeks Sıralı (Index Sequential):** anahtar değeriyle indeks bloğu aranır, oradan gerçek konuma gidilir; veritabanlarında yaygın.

12.4 İkincil Bellek ve HDD Yapısı

- **İkincil bellek:** kalıcı (non-volatile), büyük kapasiteli, CPU'ya doğrudan bağlı olmayan depolama. Türleri: **HDD** (manyetik, hareketli parçalı, ucuz), **SSD** (flash, hızlı, sessiz, pahalı), **optik diskler** (CD/DVD/Blu-ray), **USB flash**, **manyetik bant** (arşiv, çok yavaş). RAM'den ~1000 kat yavaştır (HDD).
- **Fiziksel bileşenler: Plaka (Platter)** — verinin manyetik depolandığı yüzey; **Okuma/Yazma Kafası** — her yüzey için bir kafa; **Disk Kolu (Actuator Arm)** — kafaları ize taşır; **Mil (Spindle)** — plakaları döndürür (5400–15000 RPM).
- **Mantıksal yapı: İz (Track)** — plaka üzerindeki dairesel bantlar; **Silindir (Cylinder)** — tüm plakalardaki aynı numaralı izler kümesi; **Sektör** — izin en küçük birimi (genellikle 512 B); **Blok/Cluster** — İS'nin okuduğu en küçük birim (birden çok sektör).

Disk erişim süresi formülü

Disk Erişim Süresi = Arama Süresi + Dönme Gecikmesi + Transfer Süresi
Arama (Seek Time) : kafanın doğru İZE gitmesi
Dönme Gecikmesi (Rot. Lat.) : doğru SEKTÖRÜN kafanın altına gelmesi
Transfer Süresi : verinin aktarılması

12.5 Disk Zamanlama Algoritmaları

Problem: Birden fazla işlem aynı anda diske erişmek ister; disk kontrolcüsü aynı anda **tek isteği** karşılayabilir; istekler kuyruğa alınır. **Amaçlar:** ortalama arama süresini minimize et, throughput'u maksimize et, gecikme varyansını minimize et, adil davran (açlık yaratma).

Derste kullanılan örnek: İstek kuyruğu: **82, 170, 43, 140, 24, 16, 190**; kafanın başlangıcı: **50**; disk aralığı: **0-199**.

Algoritma	Mantık	Örnek Sıra	Toplam Hareket
FCFS	İstekler geliş sırasıyla karşılanır. + Adil, açlık yok, basit. — Arama süresi optimize edilmez; kafa 'vahşi hareket' yapar.	50→82→170→43→140→24→16→190	642 silindir

Algoritma	Mantık	Örnek Sıra	Toplam Hareket
SSTF	Kafaya EN YAKIN istek seçilir. + Ortalama yanıt süresi düşer, verim artar. – AÇLIK: uzak istekler sürekli ertelenebilir; varyans yüksek. (SJF ile aynı mantık ve aynı problem!)	50→43→24→16→82→140→170→190	208 silindir (FCFS'den %68 az)
SCAN (Asansör)	Kafa bir yönde tüm istekleri karşılar; diskin UCUNA ulaşınca yön değiştirir. + Yüksek verim, düşük varyans; merkezdeki istekler avantajlı.	50→82→140→170→190→199→43→24→16	332 = (199–50)+(199–16)
C-SCAN	Kafa yalnız BİR yönde servis verir; uca gelince servis vermeden BAŞA döner. + Bekleme süreleri daha dengeli/adil; ağır yükte SCAN'den iyi.	50→82→140→170→190→199→(0'a dön)→16→24→43	391 = (199–50)+(199–0)+(43–0)
LOOK	SCAN gibi ama kafa diskin ucuna kadar GİTMEZ; SON İSTEĞE kadar gidip döner.	50→82→140→170→190→43→24→16	314 = (190–50)+(190–16)
C-LOOK	C-SCAN gibi ama son isteğe kadar gider; sonra servis vermeden EN KÜÇÜK isteğe döner. Genellikle en pratik/verimli; modern İS'lerde tercih edilir.	50→82→140→170→190→16→24→43	341 = (190–50)+(190–16)+(43–16)

► **SINAV NOTU:** Sınavda hesap sorulursa: hareket = ardışık konum farklarının MUTLAK değerlerinin toplamı. SCAN/C-SCAN'de kafa fiziksel uca (0 veya 199) gider; LOOK/C-LOOK'ta yalnızca son isteğe kadar gider. C- (circular) sürümlerde dönüş yolunda servis verilmez.

12.6 Spooling ve Buffering

- **SPOOL** = Simultaneous Peripheral Operations On-Line. CPU çok hızlı, yazıcı/disk gibi çevre birimleri çok yavaş → veriler önce **spool alanına** (diskte özel bölge) yazılır; yavaş aygıt hazır olunca sıradaki veriyi alır; işlem aygıtı beklemesiz.
- **Yazıcı örneği:** 5 belge gönderilir → hepsi spool kuyruğuna alınır → yazıcı sırayla basar → kullanıcı çalışmaya devam eder. Diğer alanlar: toplu işlem (batch), e-posta sistemleri (MTA→MUA), klavye/fare giriş tamponu.

Spooling	Buffering
İşleri kuyruklayıp uygun zamanda işlemek için bekletme.	Veri akışını yumuşatmak için KISA süreli tamponlama.
Amaç: bir kaynağı SIRAYLA paylaşmak.	Amaç: HIZ FARKINI dengelemek.
Tüm iş tamamlanınca işleme başlar.	Veri geldikçe işleme başlanabilir.
DİSK üzerinde spool alanı kullanır.	RAM'de geçici bellek (buffer) kullanır.
Birden fazla işi (job) kuyruğa alır.	Tek seferde bir iş için tampon.
Örnek: yazıcı kuyruğu, e-posta gönderme.	Örnek: video streaming, ağ iletişimi.

12.7 RAID — Bağımsız Disklerin Artıklıklı Dizisi

- Birden fazla fiziksel diski **tek mantıksal birim** olarak yönetir. **Amaç:** performansı artırmak + veri güvenliği sağlamak.
- **Temel teknikler:** **Striping (Şeritleme)** — veri bloklara bölünüp birden fazla diske paralel yazılır → hız; **Mirroring (Yansılama)** — verinin birebir kopyası başka diske yazılır → güvenlik; **Parity (Eşlik)** — XOR hesabıyla kurtarma verisi üretilir → disk arızasında veri geri yüklenir.

- **Denetleyici türleri: Donanım RAID** (en hızlı/güvenilir; ayrı kart, kendi işlemcisi); **Yazılım RAID** (İS gerçekleştirir; CPU kullanır; donanım maliyeti yok); **Firmware/Fake RAID** (yapılandırma BIOS'ta; işletim sistemi gerçekleştirir).

Seviye	Teknik	Kapasite	Dayanıklılık / Not
RAID 0	Striping (şeritleme)	$N \times B$	Yedeklilik YOK; yüksek hız; tek disk arızasında TÜM veri kaybolur.
RAID 1	Mirroring (yansılama)	(yansılama grubu sayısı) $\times B$	Bir disk bozulursa diğerinden devam edilir.
RAID 5	Striping + DAĞITIK parite (XOR)	$(N-1) \times B$	1 disk arızasına tolerans; okuma hızlı, yazma yavaş (parite hesabı); EN YAYGIN.
RAID 6	Çift parite	$(N-2) \times B$	2 disk aynı anda arızalanabilir; RAID 5'ten güvenli.
RAID 10 (1+0)	Mirroring + Striping	$(N/2) \times B$	Hem hız hem yedeklilik.

- **RAID 5 detayları** — Avantaj: okuma+yazma hızı artar; kapasite verimliliği yüksek; parite diskler arasına dağıtılır; sunucu/NAS'ta yaygın. Dezavantaj: yazma yavaş (XOR overhead); **rebuild süresi uzun ve riskli** (rebuild sırasında 2. disk arızası → veri kaybı; RAID 6 bunu çözer); **küçük yazma problemi** (small write problem).
- N: disk sayısı, B: diskteki blok sayısı. RAID değerlendirmesi: güvenilirlik | kullanılabilirlik | performans | kapasite — her seviye farklı denge sunar.

12.8 Serbest Alan Yönetimi

- **1. Bit Vektörü (Bitmap):** Her blok için 1 bit (0=boş, 1=dolu). Örn: 1111000111111001. İlk boş blok: sıfırdan farklı kelimeyi tara, içindeki ilk biti bul. + Basit, sürekli boş alan bulmak kolay; – disk büyüdükçe bitmap büyür.
- **2. Bağlı Liste:** Her boş blok bir sonraki boş bloğu gösterir. + Dinamik, verimli alan; – gezinme (traversal) için birden fazla I/O gerekebilir.
- **3. Gruplama:** n boş bloğun adresi tek blokta tutulur.
- **4. Sayma:** Sürekli boş bloklar için başlangıç adresi + adet saklanır.

13 | Hızlı Tekrar: Formüller ve Kritik Tablolar

13.1 Formüller

Formül kartı

Turnaround Time (TAT) = Completion Time - Arrival Time

Waiting Time (WT) = Turnaround Time - Burst Time

Response Time = İlk zamanlanma anı - Arrival Time

Amdahl Hızlanma = $1 / ((1 - f) + f/N)$ (f: paralel oran, N: işlemci)

Deadlock-Free : $m \geq n \times (K - 1) + 1$

Banker : Need = Max - Allocation ; güvenliyse Work += Allocation[i]

Disk Erişim Süresi = Seek + Rotational Latency + Transfer

RAID kapasiteleri : R0 = N×B | R1 = grup×B | R5 = (N-1)×B | R6 = (N-2)×B | R10 = (N/2)×B

13.2 Ezber Listeleri

- **Kritik bölge 3 kuralı:** Mutual Exclusion, Progress, Bounded Waiting.
- **Coffman 4 koşulu:** Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait — hepsi aynı anda.
- **MLFQ 5 kuralı:** 1) Öncelik büyük olan çalışır. 2) Eşitse RR. 3) Yeni iş en üst kuyruğa. 4) Kuantumunu (toplamda) bitiren bir alt kuyruğa. 5) S süresi sonunda herkes en üste (boost).
- **İşlem durumları:** New → Ready → Running → (Blocked → Ready) → Terminated.
- **Sayfa değiştirme:** FIFO, LRU, Optimal (teorik), LFU.
- **Disk zamanlama örneği sonuçları (kuyruk 82,170,43,140,24,16,190; baş 50):** FCFS 642, SSTF 208, SCAN 332, C-SCAN 391, LOOK 314, C-LOOK 341.
- **Semafor başlangıçları (klasik problemler):** P-C: mutex=1, empty=n, full=0 | R-W: mutex=1, wrt=1, readcnt=0 | Berber: customers=0, barber=0, mutex=1, freeSeats=N | Filozof: room=4, chopstick[5]={1,...}.

13.3 Karşılaştırma Mini Tabloları

Konu	A	B
Scheduler / Dispatcher	Karar verir (hangi süreç?)	Uygular (context switch + user moda geçiş)
Program / Process	Diskte pasif bayt yığını	Çalışan, canlı (adres uzayı + register + stack)
User / Kernel mode	Kısıtlı; donanıma erişemez; hata → uygulama çöker	Tam yetki; donanıma erişir; hata → sistem çöker
fork / exec	Birebir kopya; PID yeni; çocukta 0 döner	Aynı PID; belleği silip yeni programı yükler
ULT / KLT	Kütüphane yönetir; biri bloklanırsa hepsi; çoklu CPU yok	Çekirdek yönetir; gerçek paralellik; mod değişimi maliyeti
Mutex / Semafor	Nesne; kilitleme; SIKI sahiplik; lock/unlock	Tamsayı; sinyalleşme; sahiplik yok; wait/signal (P/V)
Spin / Sleep	Busy-wait; CPU israfı; kısa kilitlerde hızlı	Bekleme kuyruğu; CPU başkasına; uzun kilitlerde verimli
Deadlock / Livelock	Süreçler bloke, hareket yok	Bloke değil ama ilerleme de yok (sürekli yol verme)
Deadlock / Starvation	Karşılıklı sonsuz bekleme (döngü)	Düşük öncelikli hep atlanır; çözüm: aging

Konu	A	B
Dahili / Harici parçalanma	Blok İÇİ boşluk (sabit bölüm, sayfalama)	Bloklar ARASI dağınık boşluk (değişken bölüm); çözüm: compaction
Sayfalama / Segmentasyon	Eşit boyutlu sayfalar; harici parç. yok	Mantıksal, değişken boyutlu bölümler (kod/veri/yığın)
Mantıksal / Fiziksel adres	CPU üretir; sanal; kullanıcı görür	MMU üretir; RAM'de gerçek; doğrudan görünmez
Spooling / Buffering	Disk; iş kuyruğu; kaynağı sırayla paylaşma	RAM; hız farkını dengeleme; tek iş
TLB hit / miss	Tek bellek erişimi (hızlı)	Önce sayfa tablosu, sonra veri (2 erişim)

✓ **SON SÖZ:** Bu not; OS_1'den OS_11'e tüm slaytları ve hocanın paylaştığı 6 senkronizasyon kod dosyasını kapsar. Sınavdan önce son turda: Bölüm 3 (öncelik hesapları), 8 (kodlar), 9 (Banker örneği) ve 12.5 (disk hesapları) mutlaka elle bir kez çözülmeli. Başarılar Enes! ☐